



PUSL3190 Computing Project

Final Report

PainPal : AI-based migraine management and doctor support system

Supervisor: Diluka Wijesinghe

Name: Mathugamage D Gunawardana

Plymouth Index Number: 10953470

Degree Program: Bsc(Hons) Computer Science

Abstract

Abstract Migraines are a widespread and debilitating neurological disorder affecting over one billion individuals globally, significantly impacting quality of life, productivity, and overall well-being. Despite their prevalence, current migraine management approaches remain largely reactive, relying heavily on subjective patient self-reporting and fragmented clinical data. Existing digital tools offer basic tracking functionalities but lack predictive capabilities, personalization, and integration with clinical systems, limiting their effectiveness in proactive care. This project proposes an AI-Based Migraine Management and Doctor Support System designed to bridge the gap between patient self-management and clinical decision-making. The system integrates patient-reported data and environmental factors such as weather and sleep patterns to provide comprehensive, real-time insights. Advanced machine learning models are employed to predict high-risk migraine episodes, enabling early intervention and personalized preventive strategies. The solution consists of a crossplatform mobile application for patients and a web-based dashboard for clinicians. Patients can log symptoms, track medication adherence, and receive tailored recommendations, while clinicians gain access to structured reports, predictive analytics, and decision-support tools integrated with Electronic Health Records (EHRs). The incorporation of explainable AI ensures transparency and builds trust in clinical usage. By transforming migraine care from reactive treatment to proactive management, the proposed system aims to improve patient outcomes, enhance adherence, and support data-driven clinical decisions. Ultimately, this project contributes to more efficient, personalized, and predictive healthcare delivery in migraine management. Keywords: Migraine Management, Artificial Intelligence, Machine Learning, Predictive Analytics, Healthcare System, Clinical Decision Support, Mobile Health Applications, Explainable AI, Preventive Healthcare

Acknowledgements

I would like to express my sincere gratitude to everyone who supported me throughout the development of the **PainPal** project.

First and foremost, I am deeply thankful to my project supervisor for their continuous guidance, valuable feedback, and encouragement throughout every stage of this work. Their insights and suggestions greatly helped in shaping the direction and quality of the project.

I would also like to extend my appreciation to the academic staff of my department for providing the necessary knowledge and resources that made this project possible. Their teachings laid the foundation for both the technical and analytical aspects of this system.

Special thanks go to my workplace seniors and colleagues for their guidance, encouragement, and support during the development process. Their practical insights and experience were invaluable in improving the implementation and overall quality of the project.

I am also grateful to my friends and peers who supported me by sharing ideas, testing the system, and providing constructive feedback during development. Their support played an important role in refining the final outcome.

Finally, I sincerely thank my family for their constant encouragement, patience, and motivation throughout the entire period of this project. Their support helped me stay focused and committed to completing this work.

This project would not have been possible without the collective support of all those mentioned above.

Chapter 01 – Introduction	10
1.1 Background	10
1.2 Problem Statement.....	11
1.3 Aim and Objectives	11
1.4 Scope of the Project.....	12
1.5 Limitations.....	12
1.6 Structure of the Report.....	13
Chapter 02 – Literature Review	13
2.1 Overview of Migraine and Its Impact	13
2.2 Existing Migraine Management Systems.....	14
2.3 Artificial Intelligence in Healthcare	15
2.4 Role of Multimodal Data and Wearable Technologies.....	15
2.5 Research Gaps	16
2.6 Summary	17
Chapter 03 – System Analysis & Requirements	18
3.1 Introduction	18
3.2 Fact-Finding Techniques	18
3.2.1 Literature Review.....	18
3.2.2 Existing Application Analysis	18
3.2.3 Clinical Workflow Study	19
3.2.4 Technical Feasibility Research	19
3.3 Existing System Analysis	19
3.3.1 Traditional Methods	20
3.3.2 Digital Tracking Applications	20
3.3.3 Clinical Systems (EHRs)	20
3.4 Limitations of Existing Systems	20
3.5 Functional Requirements	21
3.5.1 Admin Requirements	21
3.5.2 Doctor Requirements.....	21
3.5.3 Patient Requirements	21
3.6 Non-Functional Requirements	21
3.6.2 Usability.....	21
3.6.3 Performance	22
3.6.4 Reliability and Availability	22
3.6.5 Scalability	22
3.7 User Requirements	22
3.8 Feasibility Study.....	22
3.8.1 Technical Feasibility	22
3.8.2 Operational Feasibility	23
3.8.3 Economic Feasibility.....	23
3.9 Summary	23
Chapter 4 – Method of Approach	24
4.1 Introduction	24
4.2 Development Methodology	24
Reasons for Choosing Agile	24
4.3 Requirement Gathering and Analysis	24
Requirement Gathering Techniques.....	25

Requirement Categorization	25
4.4 System Design Approach	25
Architecture Design	25
Database Design.....	25
User Interface Design.....	26
4.5 Technology Selection Approach	26
Frontend Technologies	26
Backend Technologies	27
Database Technology.....	27
Artificial Intelligence Technologies	27
4.6 AI/ML Development Approach	27
Data Collection	27
Data Preprocessing	27
Model Selection.....	28
Model Training and Evaluation	28
4.7 System Implementation Approach.....	28
Frontend Development.....	28
Backend Development.....	29
Integration Process	29
4.8 Testing Approach.....	29
Testing Types.....	30
AI Evaluation.....	30
4.9 Security and Privacy Approach.....	30
Security Measures	30
Privacy Considerations.....	30
4.10 Deployment and Maintenance Approach.....	30
Deployment Considerations.....	31
Maintenance Strategy.....	31
4.11 Summary	31
Chapter 05 – System Design.....	32
5.1 Introduction	32
5.2 System Architecture	32
5.2.1 Architecture Overview	32
5.2.2 Architecture Workflow.....	33
5.3 Use Case Diagram	33
5.4 Class Diagram	35
5.5 Database Design (ER Diagram)	36
5.6 User Interface Design (UI/UX)	37
5.6.1 Patient Mobile Application.....	37
5.6.2 Doctor Web Dashboard.....	41
5.7 Data Flow and Workflow Design	46
5.7.1 Data Flow Diagram (DFD)	46
5.7.2 Workflow Process	46
5.8 Security Design	47
5.8.1 Authentication and Authorization	47
5.8.2 Data Protection	47
5.8.3 Privacy Compliance	47

5.9 Integration Design	47
5.10 Summary	48
Chapter 06 – Implementation	49
6.1 Introduction	49
6.2 Development Methodology	49
6.3 Technology Stack.....	50
6.3.1 Frontend Technologies.....	50
6.3.2 Backend Technologies	50
6.3.3 Database.....	50
6.3.4 AI/ML Technologies	50
6.3.5 Other Tools	50
6.4 System Modules	51
6.4.1 Patient Mobile Application.....	51
6.4.2 Doctor Dashboard	51
6.4.3 Admin Panel.....	52
6.5 Backend and API Development	52
6.5.1 RESTful API De sign.....	52
6.5.2 Data Processing.....	52
6.5.3 Authentication	52
6.6 Database Implementation	53
6.7 AI/ML Implementation	53
6.7.1 Data Collection and Preprocessing	53
6.7.2 Model Selection	54
6.7.3 Model Training.....	54
6.7.4 Prediction Output	54
6.7.5 Explainable AI.....	54
6.8 System Integration	55
6.9 Summary	55
Chapter 07 – Testing & Evaluation	56
7.1 Introduction	56
7.2 Testing Strategy.....	56
7.3 Unit Testing	57
7.4 Integration Testing	57
7.5 System Testing.....	58
7.6 User Acceptance Testing (UAT)	58
Testing Summary.....	59
Conclusion.....	60
7.8 AI Model Evaluation.....	60
7.9 Results and Discussion	60
Model Accuracy Summary	60
XGBoost Classification Report.....	61
Chapter 08 – End Project Report	63
8.1 Introduction	63
8.2 Summary of Achievements	63
8.2.1 Development of a Cross-Platform Mobile Application	63
8.2.2 Implementation of a Web-Based Doctor Dashboard	63
8.2.3 Integration of Machine Learning Models.....	63

8.2.4 Data Integration and Management	63
8.2.5 Secure and Scalable Architecture	64
8.2.6 Improved User Experience	64
8.3 Evaluation of Objectives	64
8.4 System Performance in Real Use	65
7.5 User Feedback	66
7.6 Changes During Development.....	66
8.7 Business Value and Impact	67
8.8 Summary	68
Chapter 09 – Project Post-Mortem.....	68
9.1 Introduction	68
9.2 What Went Well	69
9.2.1 Clear Project Vision	69
9.2.2 Effective Technology Selection	69
9.2.3 Successful System Integration	69
9.2.4 Incremental Development Approach.....	69
9.2.5 User-Centered Design	69
9.3 Challenges Faced	69
8.3.1 Data Availability and Quality	69
9.3.2 AI Model Integration	70
9.3.3 Time Constraints	70
9.3.4 System Complexity.....	70
9.3.5 User Engagement Considerations.....	70
9.4 Limitations of the System	70
9.5 Lessons Learned	70
9.5.1 Importance of Planning	71
9.5.2 Value of Iterative Development	71
9.5.3 Integration Challenges	71
9.5.4 Role of Data in AI Systems.....	71
9.5.5 User Experience Matters	71
9.6 Recommendations for Future Improvements	71
9.7 Summary	71
Chapter 10 – Conclusion.....	72
10.1 Introduction.....	72
10.2 Project Summary	72
10.3 Key Contributions	73
10.4 Achievement of Objectives	73
10.5 Future Work	73
10.6 Final Remarks	74
Reference List	74
A. Migraine: Epidemiology & Management	74
B. Machine Learning & AI in Migraine.....	75
C. Wearable Technology & Migraine Forecasting.....	75
D. AI-Based Clinical Decision Support Systems.....	75
E. Explainable AI (XAI) in Healthcare	76
F. Patient Data & EHR Integration.....	76
G. Cross-Platform Mobile App Development	76

Appendix	76
Project Source Code Link	76
USER GUIDE	77
1. Introduction	77
PART A — WEB DASHBOARD (Next.js)	77
2. System Requirements	77
3. Installation (Web)	77
5. Backend Overview	78
6. Environment Variables (Web)	78
Login Credentials:	78
Hosted Web Application	78
Login Credentials:	79
PART B — MOBILE APP (Flutter)	79
1. System Requirements	79
2. Environment Variables (Web)	79
3. Installation (Mobile)	80
4.Disclaimer.....	80
TESTING	81
Mobile (Unit testing)	82
Intergration Testing.....	84
Web App.....	84
System Testing	85
User Acceptance Testing	85
Test Summary	86

List of Figures

Figure 1 - main_usecase_diagram.....	33
Figure 2 - Doctor use case.....	34
Figure 3 - patient use case	35
Figure 4 - AI Patient Summary Mobile	37
Figure 5 - Overview	37
Figure 6 - Migriane attach History Calendar Mobile	38
Figure 7 - migraine attack list.....	38
Figure 8 - migraine attack timer	39
Figure 9 - Migraine attack log	39
Figure 10 - MRI upload	40
Figure 11 - AI chatbot	40
Figure 12 - appointment scheduler	40

List of Tables

Table 1 - Model Accuracy Summary.....	60
Table 2 - XGBoost Classification Report	61
Table 3 - implimentation.....	67

Chapter 01 – Introduction

1.1 Background

Migraine is a chronic neurological disorder that affects a significant portion of the global population and is recognized as one of the leading causes of disability worldwide. Characterized by recurrent episodes of moderate to severe headache, migraines are often accompanied by symptoms such as nausea, vomiting, photophobia, phonophobia, and, in some cases, aura-related visual disturbances. These episodes can last from several hours to multiple days, severely impacting an individual's ability to perform daily activities, maintain productivity, and sustain overall well-being.

Despite the widespread prevalence and substantial burden of migraine, its management remains a complex challenge. Traditional approaches to migraine care rely heavily on retrospective self-reporting, where patients are expected to recall and describe their symptoms, triggers, and medication usage during clinical consultations. This method is inherently limited due to recall bias, inconsistencies in reporting, and the subjective nature of symptoms.

With the advancement of digital health technologies, mobile applications have emerged as tools to assist patients in tracking migraine episodes. However, most existing solutions primarily function as passive logging systems, offering limited analytical capabilities and lacking integration with clinical workflows. These systems often fail to provide predictive insights or actionable recommendations that could support proactive migraine management.

The rapid development of artificial intelligence (AI) and machine learning (ML) presents new opportunities to address these limitations. By leveraging large volumes of patient-generated data, wearable sensor inputs, and environmental factors, AI-driven systems can identify patterns, predict potential migraine triggers, and provide personalized recommendations. Furthermore, integrating such systems with clinical dashboards can enhance decision-making for healthcare professionals by offering structured and data-driven insights.

This project introduces an AI-based migraine management and doctor support system, designed to bridge the gap between patient self-management and clinical decision-making. The system integrates mobile and web technologies with machine learning models to deliver a comprehensive, proactive, and personalized migraine care solution.

1.2 Problem Statement

Current migraine management practices face several critical limitations that hinder effective diagnosis, treatment, and prevention. One of the primary challenges is the reliance on patient self-reporting, which is often inconsistent and incomplete. Patients may struggle to accurately recall details such as headache frequency, duration, intensity, triggers, and medication usage, leading to unreliable data that can affect clinical decisions.

Additionally, existing migraine tracking applications lack advanced analytical capabilities and fail to provide predictive insights. Most applications focus on retrospective data visualization rather than forecasting future migraine episodes or identifying high-risk conditions. As a result, interventions remain reactive rather than preventive.

Another significant issue is the fragmentation of data across multiple platforms. Patient-generated data from mobile applications are rarely integrated with Electronic Health Record (EHR) systems, leading to data silos that limit their usefulness in clinical settings. Healthcare professionals often rely on unstructured data stored in clinical notes, making it difficult to perform computational analysis or identify meaningful trends.

Furthermore, current systems do not effectively utilize multimodal data sources such as wearable devices, environmental conditions, and lifestyle factors. Migraines are influenced by a combination of physiological, behavioral, and environmental variables, and the lack of integrated data analysis reduces the accuracy of trigger identification and prediction.

Therefore, the problem addressed in this project can be summarized as follows:

The absence of an integrated, AI-driven migraine management system results in fragmented data handling, limited predictive capabilities, and reactive treatment approaches, ultimately leading to suboptimal patient outcomes and inefficient clinical decision-making.

1.3 Aim and Objectives

The primary aim of this project is to design and develop an AI-based migraine management and doctor support system that enhances both patient self-management and clinical decision-making through predictive analytics and data integration.

To achieve this aim, the following objectives have been defined:

- To develop a mobile application that allows patients to log migraine episodes, symptoms, medications, and lifestyle factors in a structured manner.
- To design a web-based dashboard for clinicians that provides comprehensive patient summaries, visual analytics, and AI-generated insights.
- To implement machine learning models capable of predicting high-risk migraine days using multimodal data.

- To integrate wearable sensor data and environmental information into the predictive modeling process.
- To provide personalized recommendations for preventive strategies based on individual patient patterns.
- To improve medication adherence through automated reminders and notifications.
- To ensure data security, privacy, and compliance with relevant standards such as GDPR and HIPAA.
- To enable seamless integration with existing healthcare systems and workflows.

1.4 Scope of the Project

The scope of this project encompasses the development of a comprehensive migraine management system that includes both patient-facing and clinician-facing components.

The patient module consists of a mobile application developed using Flutter, which enables users to record migraine events, track symptoms, monitor medication usage, and receive personalized insights. The application also supports data synchronization with wearable devices, allowing continuous monitoring of physiological parameters such as sleep patterns, heart rate, and physical activity.

The clinician module is implemented as a web-based dashboard using a MERN stack architecture. This dashboard provides healthcare professionals with structured patient data, visualizations of migraine trends, and AI-generated risk assessments. It facilitates efficient analysis of patient history and supports data-driven clinical decision-making.

The system also incorporates machine learning models developed using Python, models such as xgboost and resnet. These models analyze patient data, environmental factors, and behavioral patterns to predict potential migraine triggers and high-risk periods.

However, certain aspects are intentionally excluded from the scope of this project. These include genetic analysis, pharmacogenomics, and fully automated diagnosis without clinician involvement. Additionally, real-time teleconsultation features are not implemented in this phase but may be considered for future development.

1.5 Limitations

While the proposed system aims to provide a comprehensive solution for migraine management, it is subject to several limitations.

Firstly, the accuracy of predictive models depends heavily on the quality and quantity of available data. Limited datasets or incomplete user input may affect the reliability of

predictions. Secondly, the integration of wearable devices and external data sources may be constrained by compatibility issues and API limitations.

Another limitation is the potential variability in user engagement. Patients may not consistently log their symptoms or follow recommendations, which can impact the effectiveness of the system. Additionally, the system is designed as a decision-support tool rather than a diagnostic system, and clinical validation is required before deployment in real-world healthcare environments.

Finally, due to time and resource constraints, extensive large-scale testing and validation may not be fully achieved within the scope of this academic project.

1.6 Structure of the Report

This report is organized into several chapters, each focusing on a specific aspect of the project.

- Chapter 01 provides an introduction to the project, including background, problem statement, objectives, and scope.
- Chapter 02 presents a comprehensive review of existing literature related to migraine management and AI applications in healthcare.
- Chapter 03 discusses system analysis and requirements, including functional and non-functional specifications.
- Chapter 04 describes the system design, including architecture, diagrams, and database structure.
- Chapter 05 details the implementation process, technologies used, and system components.
- Chapter 06 presents testing strategies, evaluation methods, and results.
- Chapter 07 provides an end-project evaluation and analysis of outcomes.
- Chapter 08 includes a project post-mortem and reflection on challenges and lessons learned.
- Chapter 09 concludes the report and outlines future work.

Chapter 02 – Literature Review

2.1 Overview of Migraine and Its Impact

Migraine is a highly prevalent and disabling neurological condition that affects more than one billion individuals globally . It is characterized by recurrent episodes of moderate to

severe headache, often accompanied by nausea, vomiting, and sensitivity to light and sound. In some cases, migraines are preceded by aura, which includes visual or sensory disturbances. The complexity of migraine arises from its multifactorial nature, involving genetic predisposition, neurological dysfunction, hormonal changes, and environmental triggers.

The socio-economic burden of migraine is substantial. Individuals suffering from frequent migraine attacks often experience reduced productivity, absenteeism, and diminished quality of life. Students may struggle with academic performance, while working adults face challenges in maintaining consistent work output. Chronic migraine conditions can also lead to psychological issues such as anxiety and depression, further compounding the impact on overall well-being.

Despite its widespread prevalence, migraine remains underdiagnosed and undertreated. One major reason for this is the subjective nature of symptom reporting and the variability of triggers among individuals. This variability makes it difficult to apply standardized treatment approaches. Consequently, there is a growing need for personalized and data-driven solutions that can better capture the complexity of migraine patterns and support effective management strategies.

2.2 Existing Migraine Management Systems

Traditional migraine management relies heavily on patient self-reporting and clinical consultations. Patients are typically asked to describe their symptoms, triggers, and medication usage based on memory, which introduces recall bias and inaccuracies. Paper-based headache diaries have long been used as a method of tracking migraine episodes, but they are often inconvenient and prone to incomplete data entry.

With the advancement of mobile technology, several digital migraine tracking applications have been developed. These applications allow users to log headache occurrences, pain intensity, duration, and potential triggers. Some applications also provide visualizations such as charts and graphs to help users understand their migraine patterns over time.

However, analysis of existing applications reveals several limitations. Most systems function as passive data collection tools rather than intelligent support systems. They focus on retrospective tracking rather than proactive management. While users can view historical trends, they are not provided with predictive insights that could help prevent future attacks.

Another major limitation is the lack of integration with clinical systems. Data collected through mobile applications are rarely shared with healthcare providers in a structured and meaningful way. As a result, clinicians continue to rely on verbal reports during consultations, limiting the usefulness of digital tracking tools in clinical decision-making.

- Furthermore, many existing systems do not incorporate contextual data such as sleep patterns, stress levels, or environmental conditions. Migraines are influenced by a combination of physiological and environmental factors, and ignoring these variables

reduces the effectiveness of trigger identification. These limitations highlight the need for more advanced systems that go beyond basic tracking and provide integrated, intelligent, and clinically relevant solutions.

2.3 Artificial Intelligence in Healthcare

- Artificial Intelligence (AI) has emerged as a transformative technology in healthcare, offering new possibilities for diagnosis, prediction, and personalized treatment. AI techniques such as machine learning and deep learning enable systems to analyze large volumes of data, identify patterns, and generate insights that would be difficult for humans to detect.
- In the context of neurological disorders, AI has been applied to various tasks, including disease prediction, medical imaging analysis, and treatment optimization. Machine learning models can process structured data such as patient symptoms, medical history, and lifestyle factors to predict the likelihood of disease occurrence or progression. Similarly, deep learning models have shown significant success in analyzing medical images, such as MRI scans, for the detection of abnormalities.
- One of the key advantages of AI in healthcare is its ability to support personalized medicine. By analyzing individual patient data, AI systems can generate tailored recommendations that consider unique patterns and risk factors. This is particularly relevant for migraine management, where triggers and symptoms vary significantly between individuals.
- However, the adoption of AI in clinical settings also presents challenges. One major concern is the lack of transparency in many AI models, often referred to as the “black box” problem. Clinicians may be hesitant to rely on AI-generated predictions if they do not understand how the results were derived. This has led to increased interest in Explainable AI (XAI), which aims to provide clear and interpretable insights into model behavior.
- Another challenge is data quality and availability. AI models require large, high-quality datasets for training and validation. Inconsistent or incomplete data can reduce model accuracy and reliability. Additionally, privacy and security concerns must be addressed when handling sensitive healthcare data.
- Despite these challenges, AI remains a promising tool for improving healthcare outcomes. Its ability to analyze complex, multimodal data makes it particularly suitable for conditions like migraine, where multiple factors contribute to disease patterns.

2.4 Role of Multimodal Data and Wearable Technologies

- Recent advancements in wearable technology and IoT devices have enabled continuous monitoring of physiological and environmental parameters. Devices such as smartwatches and fitness trackers can collect data on heart rate, sleep quality, physical activity, and stress levels. Environmental data, including weather conditions, temperature, humidity, and air quality, can also be integrated through external APIs.

- The use of multimodal data is particularly important in migraine management. Research indicates that migraines are often triggered by a combination of factors rather than a single cause. For example, a migraine attack may be influenced by poor sleep, high stress, and changes in weather conditions simultaneously. Traditional tracking methods that focus only on symptoms fail to capture these complex interactions.
- By integrating multiple data sources, systems can gain a more comprehensive understanding of migraine patterns. Machine learning models can analyze correlations between different variables and identify hidden relationships that may not be immediately apparent. This enables more accurate prediction of migraine episodes and more effective identification of triggers.
- Wearable devices also support real-time data collection, reducing reliance on manual input. This improves data accuracy and consistency while minimizing user effort. However, challenges remain in terms of device compatibility, data standardization, and integration with existing systems.
- Overall, the incorporation of multimodal data and wearable technologies represents a significant advancement in migraine management. It enables a shift from isolated symptom tracking to holistic, data-driven analysis, which is essential for effective prediction and prevention.

2.5 Research Gaps

- Despite the availability of digital tools and advancements in AI, several gaps remain in current migraine management solutions.
- Firstly, there is a lack of integration between patient-generated data and clinical systems. Most applications operate in isolation, preventing seamless data sharing with healthcare providers. This limits their usefulness in clinical decision-making and reduces the potential benefits of digital health technologies.
- Secondly, existing systems lack predictive capabilities. While some applications provide basic trend analysis, they do not utilize advanced machine learning techniques to forecast future migraine episodes. As a result, patients and clinicians are unable to take preventive actions based on early warnings.
- Another significant gap is the limited use of multimodal data. Many systems rely solely on user input and do not incorporate wearable or environmental data. This reduces the accuracy of trigger identification and prediction models.
- Additionally, there is a lack of explainability in AI-based systems. Even when predictive models are used, they often do not provide clear explanations for their outputs. This reduces trust among clinicians and hinders adoption in real-world healthcare settings.
- Finally, user engagement remains a challenge. Many applications require consistent manual input, which can be burdensome for users, especially during migraine episodes. Systems that do not prioritize usability and low-friction interaction are less likely to be adopted and maintained over time.

- These gaps highlight the need for a comprehensive solution that integrates data sources, provides predictive insights, ensures explainability, and enhances user engagement.

2.6 Summary

- This chapter reviewed existing literature related to migraine management, digital health applications, artificial intelligence, and multimodal data integration. It is evident that while significant progress has been made in developing tools for migraine tracking and analysis, current solutions remain limited in their ability to provide proactive, personalized, and clinically integrated care.
- Traditional methods rely heavily on subjective reporting, while existing digital applications focus primarily on retrospective data collection. AI technologies offer promising capabilities for prediction and personalization but face challenges related to transparency, data quality, and clinical integration.
- The use of wearable devices and multimodal data presents new opportunities for comprehensive analysis and improved prediction accuracy. However, current systems do not fully leverage these capabilities.
- Based on the identified research gaps, there is a clear need for an integrated, AI-driven migraine management system that combines predictive analytics, multimodal data integration, explainable AI, and clinical support features. The proposed system aims to address these limitations and contribute to the advancement of personalized and proactive migraine care.

Chapter 03 – System Analysis & Requirements

3.1 Introduction

System analysis is a critical phase in the development of any software system, as it establishes a clear understanding of user needs, system functionality, and operational constraints. For the proposed AI-Based Migraine Management and Doctor Support System (PainPal), the analysis phase focused on identifying the limitations of current migraine management approaches and defining requirements for a more efficient, integrated, and intelligent solution.

This chapter presents the fact-finding techniques used to gather system requirements, analyzes existing systems and their shortcomings, and defines both functional and non-functional requirements. Additionally, it outlines the feasibility considerations that justify the development of the proposed system.

3.2 Fact-Finding Techniques

To ensure that the system is designed based on real-world needs and validated knowledge, multiple fact-finding techniques were employed.

3.2.1 Literature Review

A comprehensive literature review was conducted to understand the current state of migraine management and the role of artificial intelligence in healthcare. Academic journals, medical publications, and technical research papers were analyzed to identify trends, challenges, and opportunities in migraine care.

The review highlighted several key findings:

- Migraine management is largely reactive and dependent on subjective reporting.
- AI and machine learning have strong potential in predictive healthcare.
- Explainable AI is essential for clinical adoption.
- Digital health applications often lack integration with clinical workflows.

These insights formed the theoretical foundation for the proposed system.

3.2.2 Existing Application Analysis

Several migraine tracking and health monitoring applications were analyzed to evaluate their features, usability, and limitations. The analysis focused on:

- Symptom logging capabilities
- Data visualization features
- Integration with wearable devices
- Clinical usability

It was observed that most applications provide only basic tracking features and do not support predictive analytics or advanced personalization. Additionally, integration with healthcare systems is minimal, limiting their usefulness in clinical settings.

3.2.3 Clinical Workflow Study

To understand how migraine data is handled in real healthcare environments, clinical workflows and Electronic Health Record (EHR) systems were studied. This analysis revealed that:

- Migraine data is often stored as unstructured text.
- Clinicians rely heavily on patient recall.
- There is limited use of structured data for analysis.

These findings emphasized the need for structured data collection and integration with clinical systems.

3.2.4 Technical Feasibility Research

Experimental implementation of key technologies was carried out to assess feasibility. Technologies such as Flutter, Node.js, MongoDB, and Python-based machine learning frameworks were tested.

The results confirmed that:

- Cross-platform mobile applications can be efficiently developed using Flutter.
- Next js provides quick production ready MVP with heighest backend performance available copared to other frameworks.
- Python supports robust AI/ML model development.
- Cloud-based databases enable scalable and secure data storage.

This validation ensured that the proposed system is technically achievable.

3.3 Existing System Analysis

Current migraine management systems can be broadly categorized into three types:

3.3.1 Traditional Methods

Traditional approaches include paper-based headache diaries and verbal reporting during clinical consultations. These methods rely heavily on patient memory and are prone to inaccuracies.

3.3.2 Digital Tracking Applications

Mobile applications have improved accessibility by allowing users to log migraine events digitally. These apps typically include:

- Symptom tracking
- Trigger identification
- Basic visualizations

However, they lack predictive capabilities and clinical integration.

3.3.3 Clinical Systems (EHRs)

Electronic Health Record systems store patient data but often in unstructured formats. Migraine-related data is usually recorded as text notes, making it difficult to analyze computationally.

3.4 Limitations of Existing Systems

The analysis of existing systems revealed several critical limitations:

- **Reliance on Self-Reporting:** Patients often provide incomplete or inaccurate data due to recall bias.
- **Data Fragmentation:** Information is scattered across multiple platforms, leading to inefficiencies.
- **Lack of Predictive Analytics:** Current systems do not forecast future migraine episodes.
- **Limited Personalization:** Recommendations are generic and not tailored to individual users.
- **Poor Clinical Integration:** Data collected from apps is not effectively used in clinical decision-making.
- **Absence of Multimodal Data:** Most systems ignore wearable and environmental data.
- **Lack of Explainability:** AI-based systems do not provide transparent insights.

These limitations justify the need for a more advanced, integrated solution.

3.5 Functional Requirements

Functional requirements define the core functionalities of the system based on user roles.

3.5.1 Admin Requirements

- The system shall allow the admin to create, update, and delete user accounts.
- The admin shall manage doctors, patients, and clinic relationships.
- The admin shall monitor system operations and maintain system integrity.
- The admin shall have full access to all system data.

3.5.2 Doctor Requirements

- The system shall allow doctors to view patient profiles and medical history.
- Doctors shall access migraine logs and medication records.
- The system shall enable doctors to create clinical notes and manage appointments.
- Doctors shall view AI-generated insights and summaries.
- The system shall support treatment monitoring and outcome tracking.

3.5.3 Patient Requirements

- The system shall allow patients to log migraine events with structured inputs.
- Patients shall view their migraine history and trends.
- The system shall provide predicted migraine types based on input data.
- Patients shall log medication usage and symptom progression.
- The system shall provide reminders and notifications for medication adherence.

3.6 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

3.6.1 Security and Access Control

- The system shall implement Role-Based Access Control (RBAC).
- Authentication mechanisms shall ensure secure access to the system.
- Unauthorized access attempts shall be restricted.
- Sensitive data shall be encrypted and securely stored.

3.6.2 Usability

- The system shall provide intuitive interfaces for all user roles.
- Patient interfaces shall be optimized for mobile devices.
- Doctor and admin dashboards shall support efficient data review.
- The system shall minimize user input effort through structured forms.

3.6.3 Performance

- The system shall handle multiple concurrent users efficiently.
- API responses shall be optimized for low latency.
- Data retrieval and processing shall be performed in real-time where possible.

3.6.4 Reliability and Availability

- The system shall ensure high availability with minimal downtime.
- Data shall be consistently stored and backed up.
- The system shall support offline data entry and later synchronization.

3.6.5 Scalability

- The system shall support increasing numbers of users and data volume.
- Cloud infrastructure shall enable dynamic resource allocation.

3.7 User Requirements

The system is designed to meet the needs of three primary user groups:

Patients

- Easy-to-use mobile interface
- Quick symptom logging
- Personalized insights and predictions
- Medication reminders

Doctors

- Access to structured patient data
- Visualization of migraine trends
- AI-assisted decision support
- Efficient patient management tools

Administrators

- User and system management
- Monitoring and maintenance
- Data governance

3.8 Feasibility Study

3.8.1 Technical Feasibility

The system is technically feasible due to the availability of modern development frameworks and tools. Technologies such as Flutter, Node.js, MongoDB, and Python provide a strong

foundation for development. Machine learning models can be effectively implemented using established libraries such as TensorFlow and Scikit-learn.

3.8.2 Operational Feasibility

The system aligns well with real-world workflows. Patients can easily log data using mobile devices, while doctors can access structured information through a web dashboard. The system enhances existing processes rather than replacing them, making adoption more practical.

3.8.3 Economic Feasibility

The project is economically viable due to the use of open-source technologies and scalable cloud services. Initial development costs are manageable, and operational costs can be controlled through efficient resource usage. The long-term benefits include improved healthcare efficiency and reduced manual effort.

3.9 Summary

This chapter analyzed the current migraine management landscape and identified key limitations in existing systems. Through various fact-finding techniques, the requirements for a more advanced and integrated system were established.

The proposed system addresses these limitations by incorporating structured data collection, predictive analytics, multimodal data integration, and clinical support features. Functional and non-functional requirements were defined to guide system development, while feasibility analysis confirmed that the project is practical and achievable.

Chapter 4 – Method of Approach

4.1 Introduction

The Method of Approach describes the systematic process followed during the development of the AI-Based Migraine Management and Doctor Support System (PainPal). It outlines the methodologies, tools, technologies, and development practices used to ensure the successful completion of the project.

The project adopted a structured and iterative development approach that combined software engineering principles, artificial intelligence techniques, and user-centered design practices. The methodology was designed to support continuous improvement, effective integration of system components, and efficient management of project requirements.

4.2 Development Methodology

An **Agile Software Development Methodology** was selected for this project due to its flexibility, adaptability, and suitability for iterative system development. Agile enables continuous development and testing throughout the project lifecycle, allowing requirements and features to evolve over time.

The project was divided into multiple development iterations, where each iteration focused on a specific module or functionality of the system. Feedback obtained during testing and evaluation phases was incorporated into subsequent iterations to improve system quality and usability.

Reasons for Choosing Agile

- Supports incremental development
- Allows continuous testing and improvement
- Facilitates handling changing requirements
- Improves project flexibility and adaptability
- Encourages modular development and integration

The Agile approach ensured that critical functionalities such as symptom tracking, AI prediction, and dashboard visualization could be developed independently and integrated progressively.

4.3 Requirement Gathering and Analysis

The initial phase of the project focused on understanding the problem domain and identifying user requirements.

Requirement Gathering Techniques

Several techniques were used to gather and analyze requirements:

- Literature review of migraine management systems
- Analysis of existing healthcare applications
- Research on artificial intelligence in healthcare
- Study of migraine-related medical and environmental factors

These activities helped identify the limitations of existing systems and establish the core functionalities required for the proposed solution.

Requirement Categorization

Requirements were categorized into:

- Functional requirements
- Non-functional requirements
- Security requirements
- AI-related requirements

This categorization ensured a structured approach to system design and implementation.

4.4 System Design Approach

After requirement analysis, the system architecture and design models were created.

Architecture Design

A multi-tier architecture was adopted to separate the system into distinct layers:

- Presentation layer
- Backend application layer
- Database layer
- AI/ML layer

This modular design improved maintainability, scalability, and integration.

Database Design

MongoDB was selected as the database management system due to its flexibility and ability to handle semi-structured healthcare data.

The database was designed to store:

- Patient information
- Migraine logs
- Medication records
- AI prediction results
- Clinical notes

User Interface Design

User-centered design principles were applied to ensure usability and accessibility.

Mobile Application

The patient mobile application was designed with:

- Simple navigation
- Quick symptom logging
- Visual trend analysis
- Notification support

Web Dashboard

The clinician dashboard focused on:

- Data visualization
- Efficient patient management
- Access to AI-generated insights

Figma was used to prototype and refine the interface designs before implementation.

4.5 Technology Selection Approach

Technologies were selected based on scalability, compatibility, performance, and ease of integration.

Frontend Technologies

- **Flutter** was selected for mobile application development due to its cross-platform capabilities.
- **React.js** was used for the web dashboard because of its component-based architecture and responsive UI support.

Backend Technologies

- Next Js api were used to develop RESTful APIs and backend services.
- These technologies support asynchronous processing and scalable server-side operations.

Database Technology

- **MongoDB** was selected because it supports flexible document-based storage, making it suitable for evolving healthcare datasets.

Artificial Intelligence Technologies

- **Python** was used for machine learning implementation due to its extensive AI ecosystem.
- Libraries such as **Scikit-learn** and **TensorFlow** were used for model development and evaluation.

4.6 AI/ML Development Approach

Artificial intelligence was implemented to provide predictive capabilities and intelligent insights.

Data Collection

The AI models utilized data from:

- Patient symptom logs
- Lifestyle-related information
- Environmental conditions

Data Preprocessing

Before training the models, the collected data underwent preprocessing steps such as:

- Handling missing values
- Data normalization
- Feature extraction

- Data transformation

These steps improved model quality and consistency.

Model Selection

Several machine learning algorithms were evaluated, including:

- Random Forest
- Decision Trees
- XGBoost

The selected models were chosen based on their classification performance and suitability for structured healthcare data.

Model Training and Evaluation

The dataset was divided into training and testing datasets using standard machine learning practices.

The performance of the models was evaluated using metrics such as:

- Accuracy
- Precision
- Recall
- F1 Score

The evaluation ensured that the selected models provided reliable prediction performance.

4.7 System Implementation Approach

The implementation phase followed a modular and incremental strategy.

Frontend Development

The mobile application and web dashboard were developed independently and connected through APIs.

Patient Mobile Application

Implemented features included:

- Authentication
- Migraine logging
- Medication tracking
- Prediction visualization

Doctor Dashboard

Implemented features included:

- Patient data visualization
- Clinical note management
- AI insight display

Backend Development

The backend was implemented using RESTful architecture.

Key backend functionalities included:

- User authentication
- Data validation
- Database communication
- AI model integration
- API management

Integration Process

System integration involved connecting:

- Mobile app ↔ Backend
- Backend ↔ Database
- Backend ↔ AI module
- Dashboard ↔ Backend

Testing was performed during each integration stage to ensure smooth communication between components.

4.8 Testing Approach

A comprehensive testing strategy was implemented throughout development.

Testing Types

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing (UAT)

Testing ensured:

- Functional correctness
- System reliability
- Performance stability
- User satisfaction

AI Evaluation

Machine learning models were evaluated separately using classification metrics to validate prediction accuracy and reliability.

4.9 Security and Privacy Approach

Since healthcare systems manage sensitive patient data, security was prioritized during development.

Security Measures

- JWT-based authentication
- Role-Based Access Control (RBAC)
- Secure API communication using HTTPS
- Data encryption mechanisms

Privacy Considerations

The system design considered healthcare data privacy principles and restricted unauthorized access to sensitive information.

4.10 Deployment and Maintenance Approach

The system was designed to support future deployment in cloud-based environments.

Deployment Considerations

- Scalable infrastructure
- API-based architecture
- Modular services

Maintenance Strategy

The modular architecture allows:

- Easy updates
- Feature enhancements
- AI model improvements
- Bug fixing and optimization

4.11 Summary

This chapter described the overall method of approach followed during the development of the AI-Based Migraine Management and Doctor Support System. An Agile methodology combined with modular system design enabled efficient development and integration of mobile, web, backend, and AI components.

The chosen technologies and development practices ensured scalability, maintainability, and usability. Additionally, the integration of AI techniques introduced predictive capabilities that enhance migraine management and clinical support.

Overall, the adopted approach provided a structured and effective framework for successfully implementing the proposed system.

Chapter 05 – System Design

5.1 Introduction

System design is a crucial phase in software development that transforms system requirements into a structured architecture and detailed design models. For the AI-Based Migraine Management and Doctor Support System (PainPal), the design focuses on creating a scalable, secure, and efficient solution that integrates mobile applications, web dashboards, and machine learning components.

This chapter presents the overall system architecture, design diagrams, database structure, user interface design, and workflow processes. The design ensures that the system meets both functional and non-functional requirements while maintaining flexibility for future enhancements.

5.2 System Architecture

The proposed system follows a **multi-tier architecture** that separates concerns into distinct layers, ensuring modularity and scalability.

5.2.1 Architecture Overview

The system consists of four primary layers:

1. Presentation Layer

- Mobile application (Flutter) for patients
- Web dashboard (Next js) for doctors and administrators

2. Application Layer (Backend)

- Node.js and Next Js SSR backend
- RESTful API services
- Authentication and authorization services

3. Data Layer

- MongoDB database with Prisma
- Structured storage for patient records, logs, and predictions

4. AI/ML Layer

- Python-based machine learning models
- Prediction engine for migraine risk assessment
- Data preprocessing and feature extraction modules

5.2.2 Architecture Workflow

The workflow of the system is as follows:

- Patients input migraine-related data through the mobile application.
- Data is transmitted securely to the backend via REST APIs.
- The backend processes and stores data in the database.
- The AI module analyzes the data and generates predictions.
- Results are sent back to the application and displayed to users.
- Doctors access patient data and insights through the web dashboard.

This layered approach ensures **separation of concerns**, making the system easier to maintain and scale.

5.3 Use Case Diagram

The use case diagram represents interactions between system users and the system.

Figure 1 - main_usecase_diagram

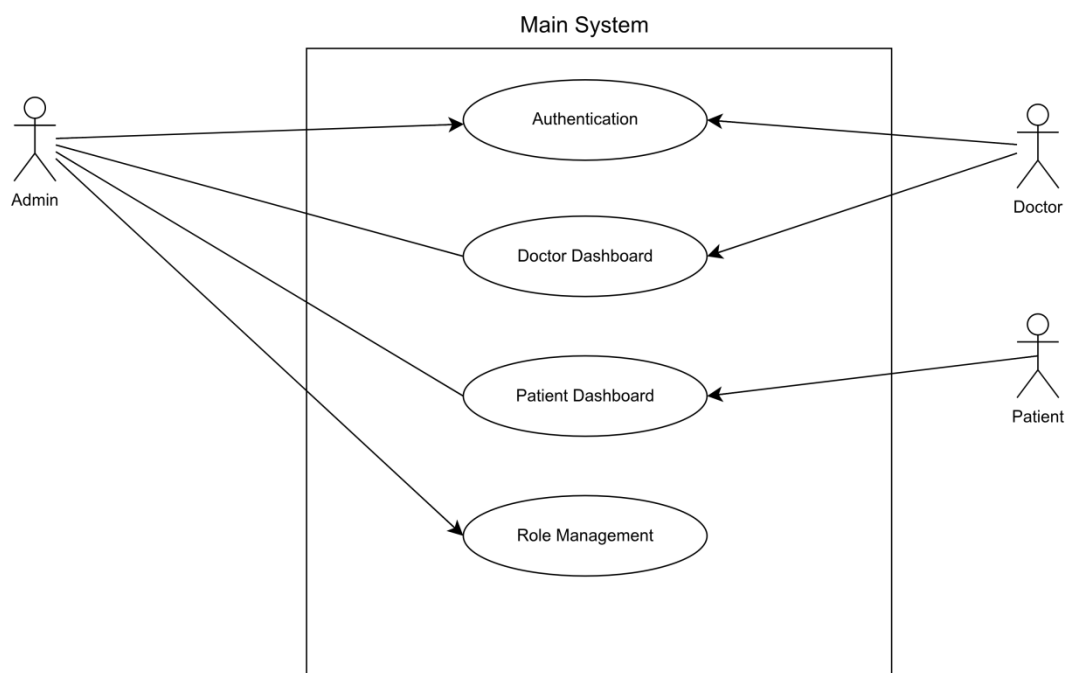


Figure 2 - Doctor use case

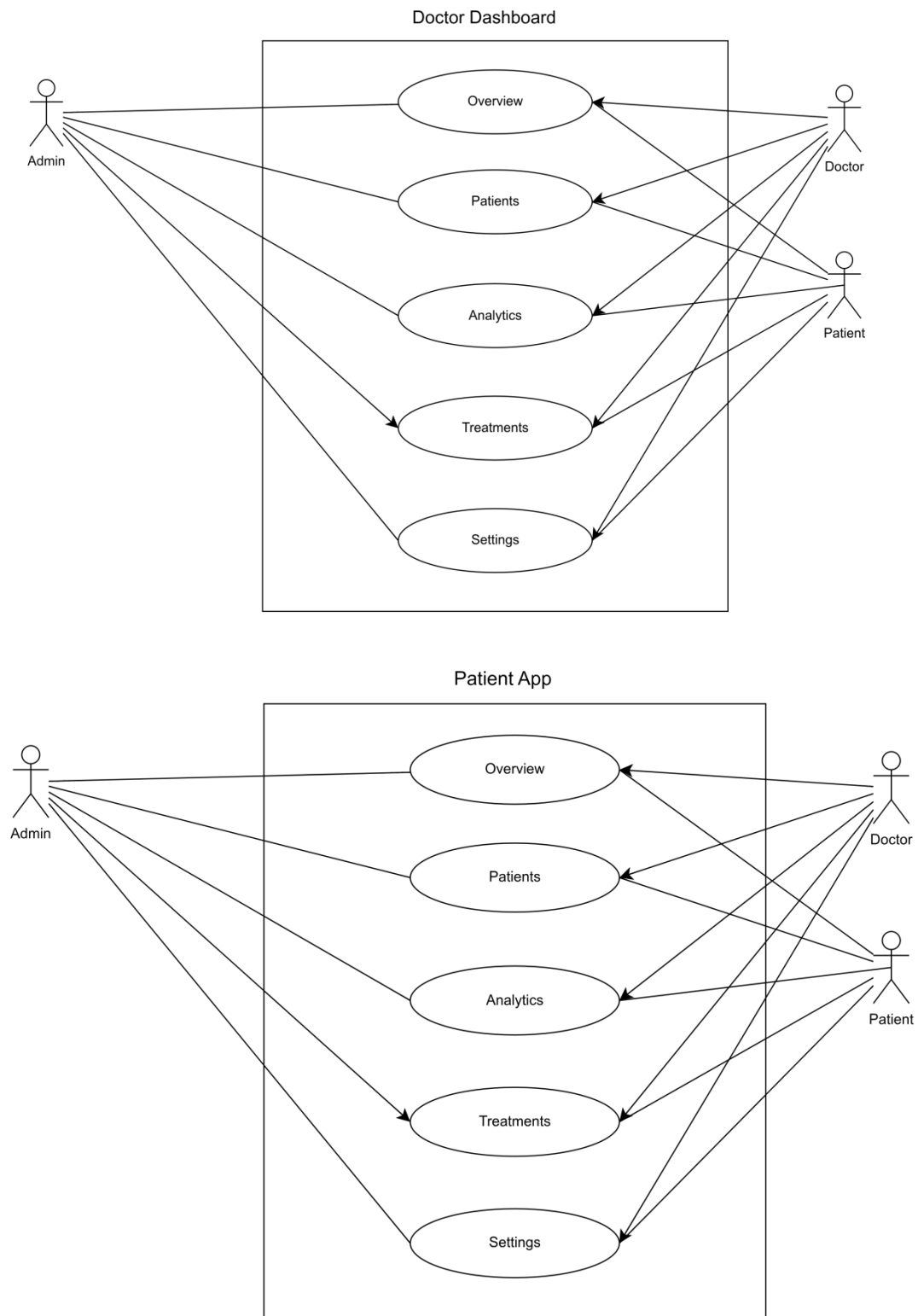


Figure 3 - patient use case

Explanation

The use case diagram highlights role-based interactions and ensures that system functionalities are aligned with user needs. It also supports the implementation of Role-Based Access Control (RBAC).

5.4 Class Diagram

The class diagram defines the structure of the system by representing classes, attributes, and relationships.

Main Classes

- **User**
 - userId
 - name
 - email
 - role
- **Patient (extends User)**
 - medicalHistory
 - migraineLogs
- **Doctor (extends User)**
 - specialization
 - assignedPatients
- **MigraineLog**
 - logId
 - date
 - duration

- intensity
 - triggers
- **Medication**
 - medicationId
 - name
 - dosage
- **Prediction**
 - predictionId
 - riskLevel
 - recommendedActions

Relationships

- A patient can have multiple migraine logs
- A doctor can manage multiple patients
- Each migraine log can generate one or more predictions

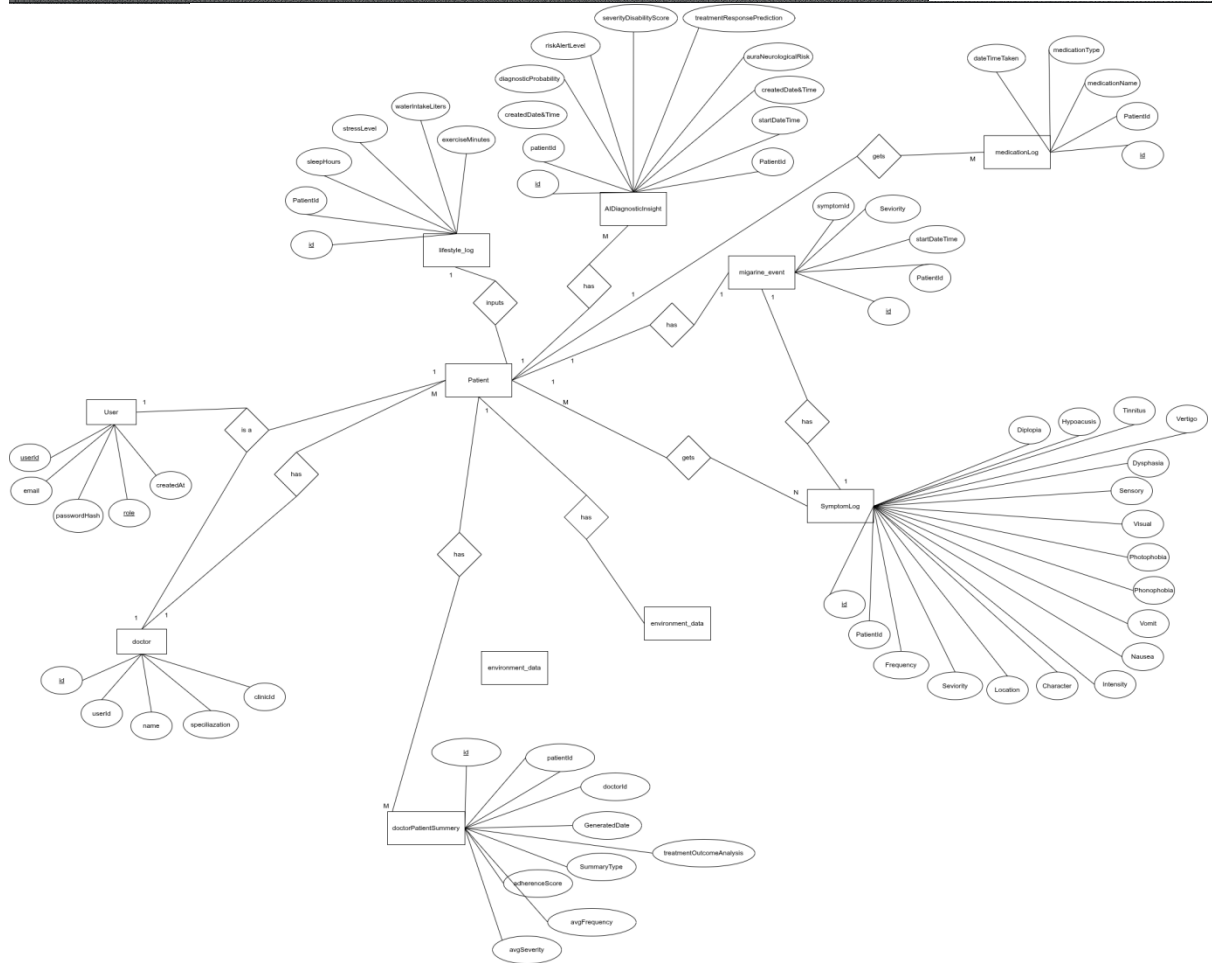
Explanation

The class diagram ensures proper data organization and supports object-oriented design principles such as inheritance and encapsulation.

5.5 Database Design (ER Diagram)

The database is designed using a **NoSQL approach (MongoDB)** to handle flexible and scalable data storage.





Explanation

The NoSQL design allows dynamic schema updates, which is essential for handling evolving healthcare data. It also supports high scalability and performance.

5.6 User Interface Design (UI/UX)

The system includes two primary interfaces:

5.6.1 Patient Mobile Application

Designed with a focus on usability and accessibility:

Figure 4 - AI Patient Summary Mobile

Figure 5 - Overview Mobile

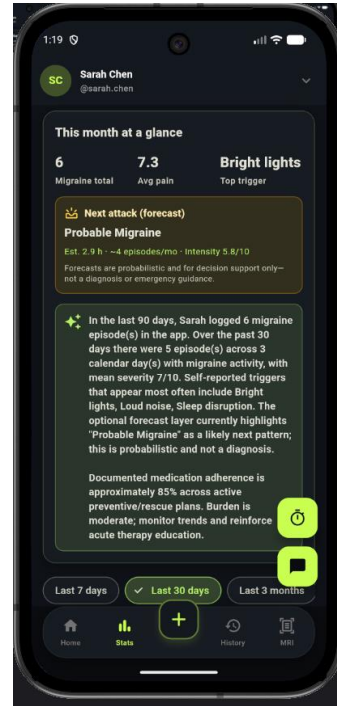
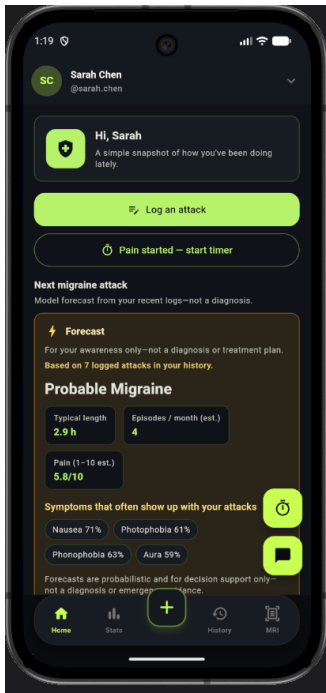


Figure 6 - Migraine attack History Calendar Mobile



Figure 7 - migraine attack list

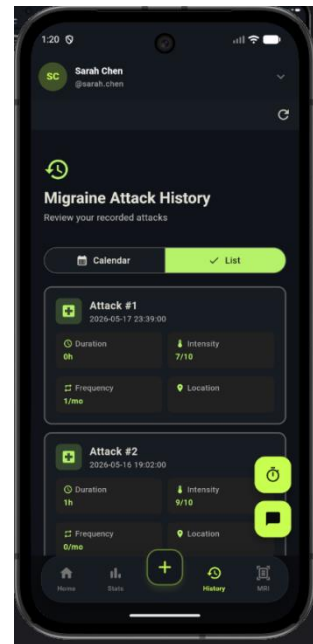


Figure 8 - migraine attack timer

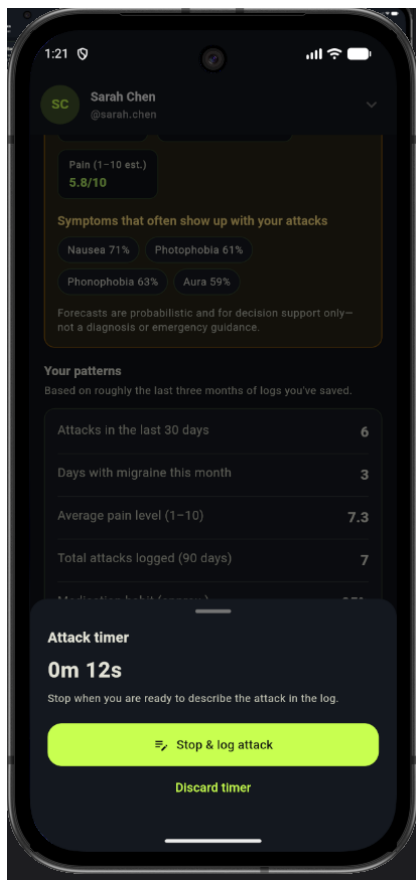


Figure 9 - Migraine attack log

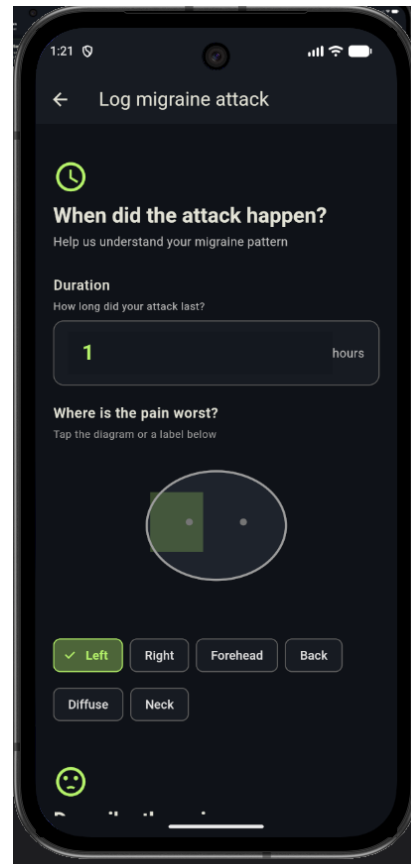


Figure 10 - MRI upload

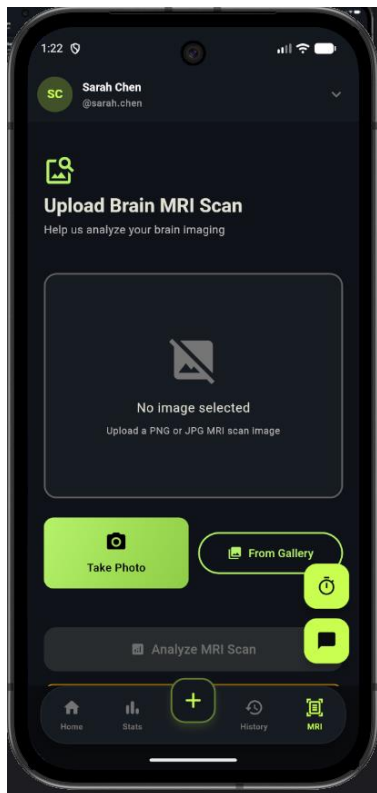


Figure 11 - AI chatbot

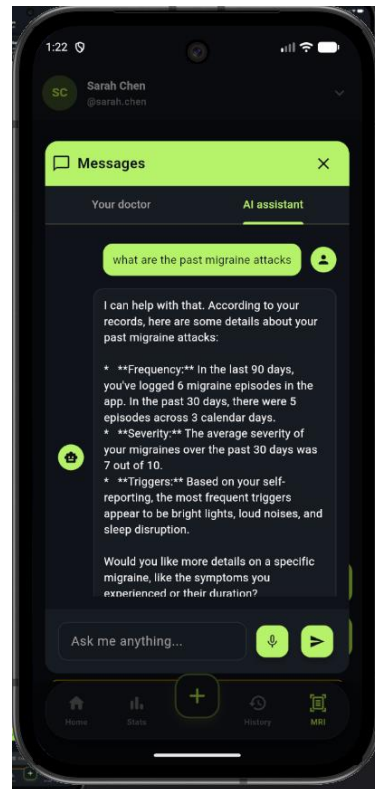
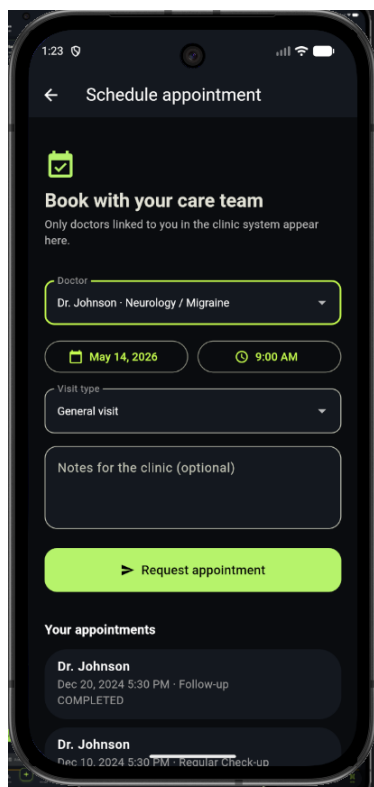


Figure 12 - appoinment scheduler

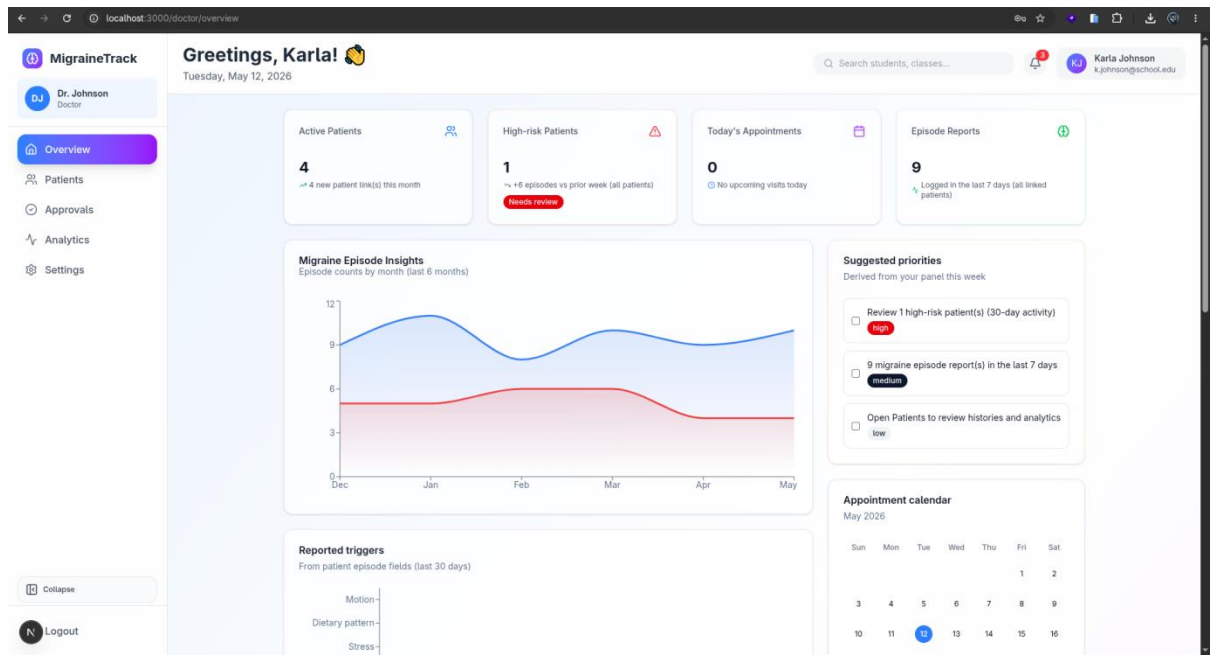


- Simple and intuitive navigation
- Quick symptom logging interface
- Visual charts for migraine trends
- Notification system for reminders

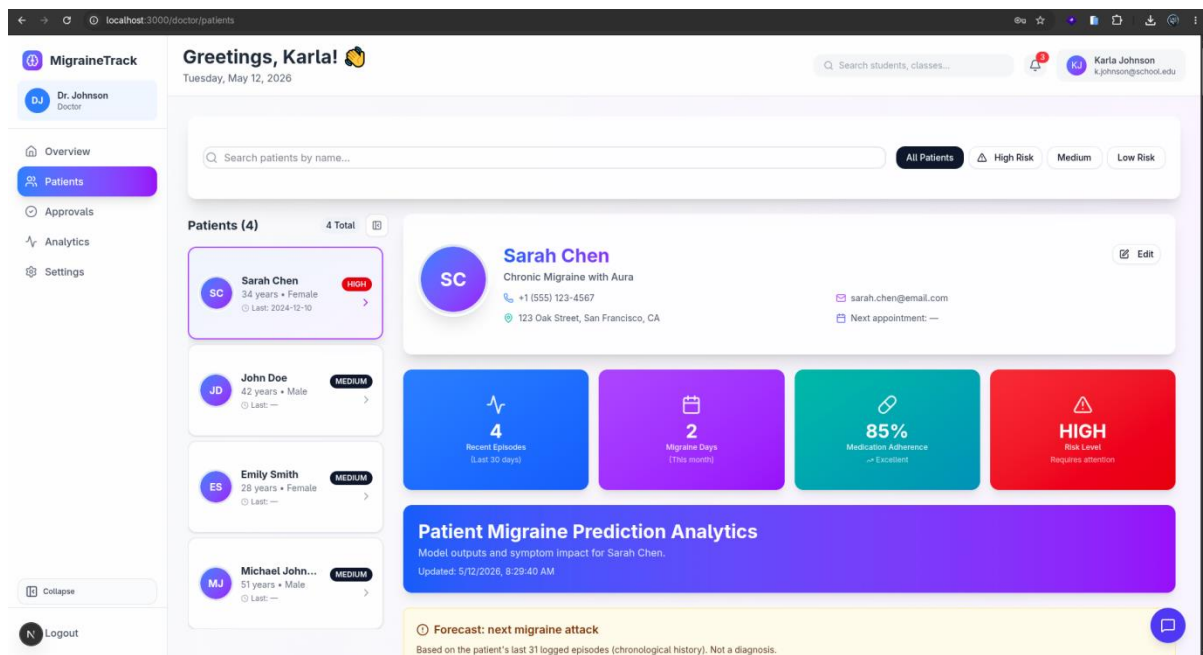
5.6.2 Doctor Web Dashboard

Designed for efficient data analysis:

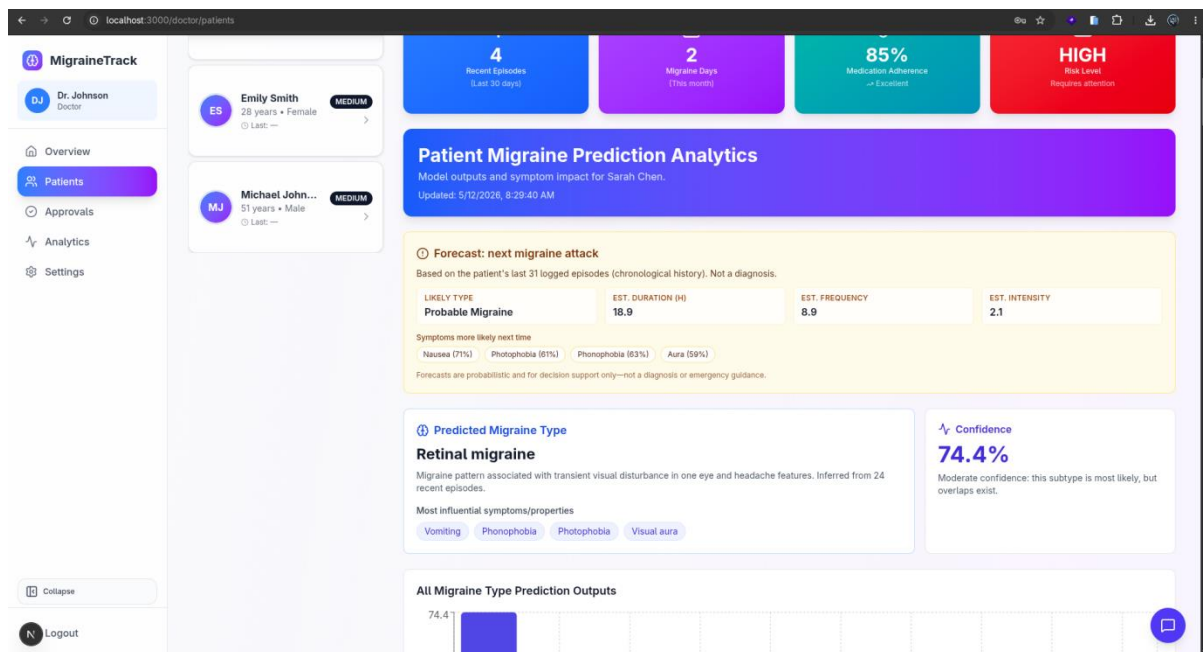
- Doctor Overview Dashboard



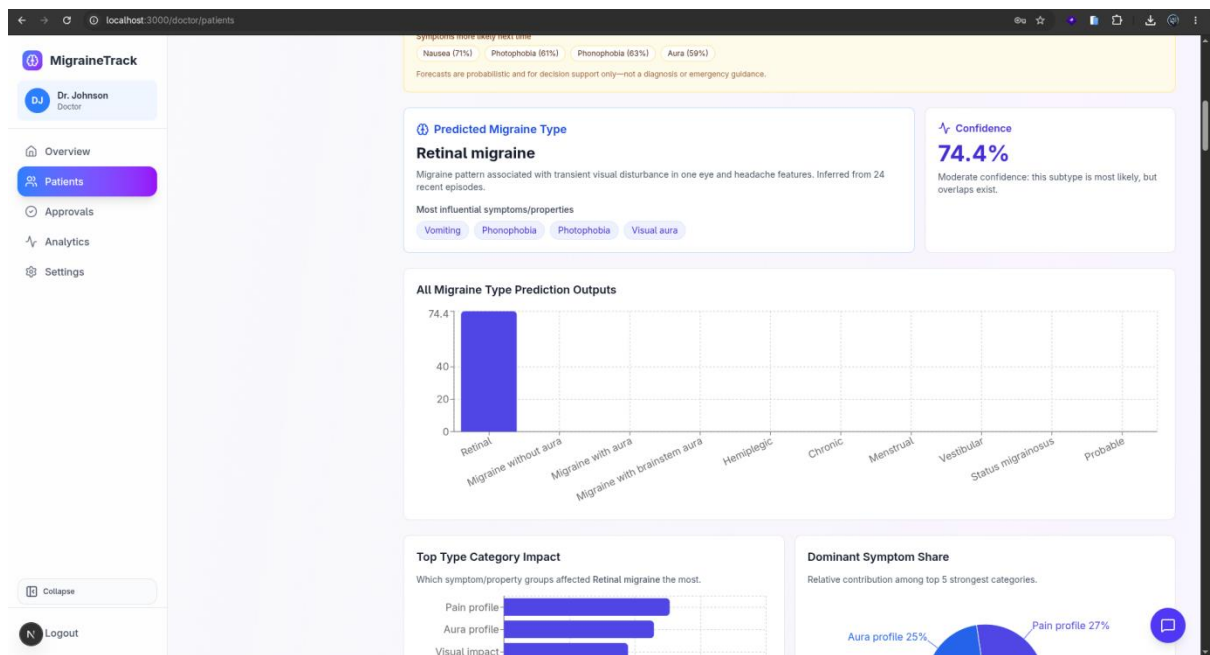
- Patient Dashboard
 - Patient Overview



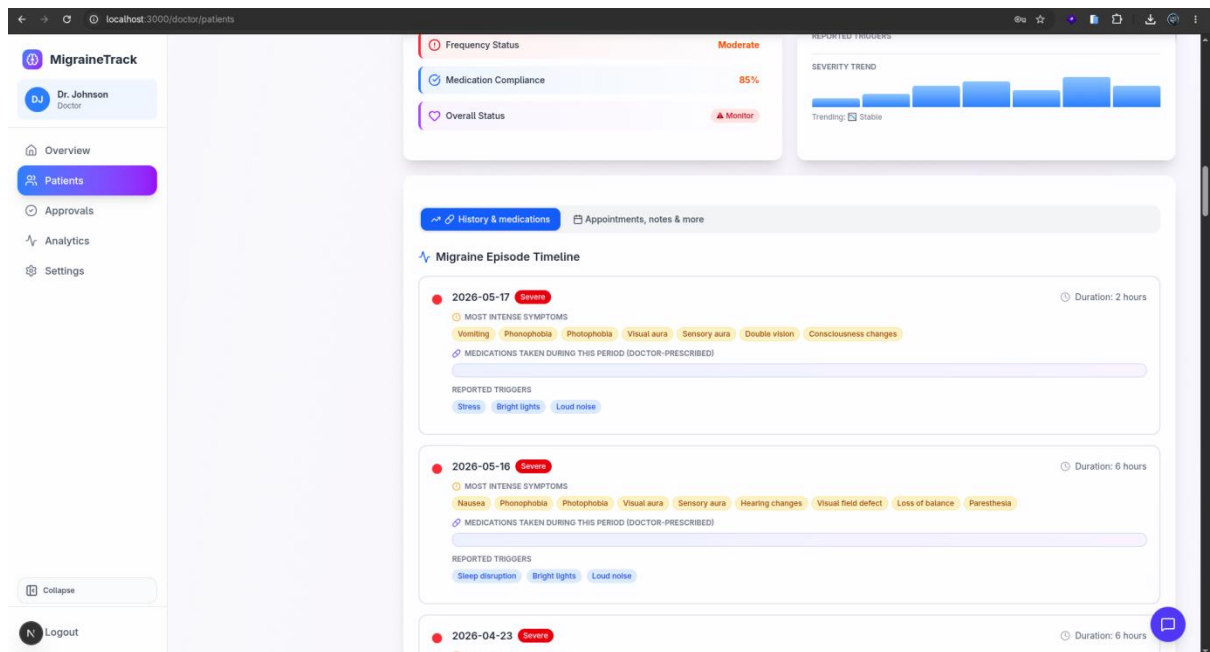
○ Forcast Migraine Prediction (Next Attack Prediction)



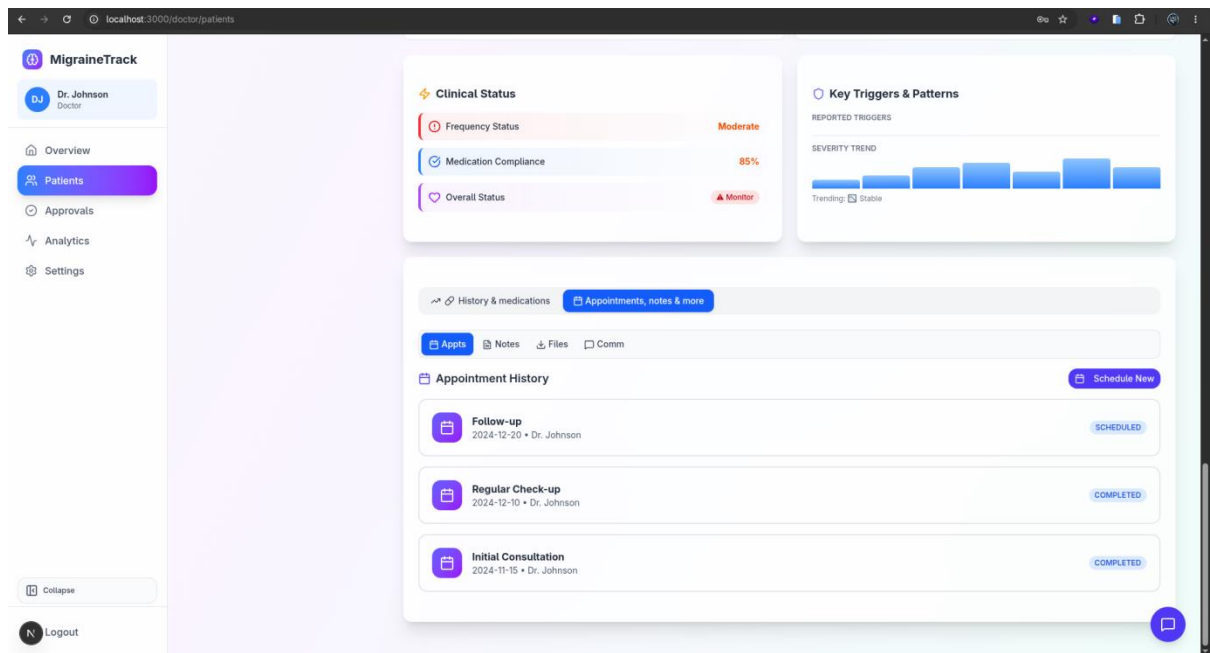
○ Patient Migraine Type Prediction and Confidence Score



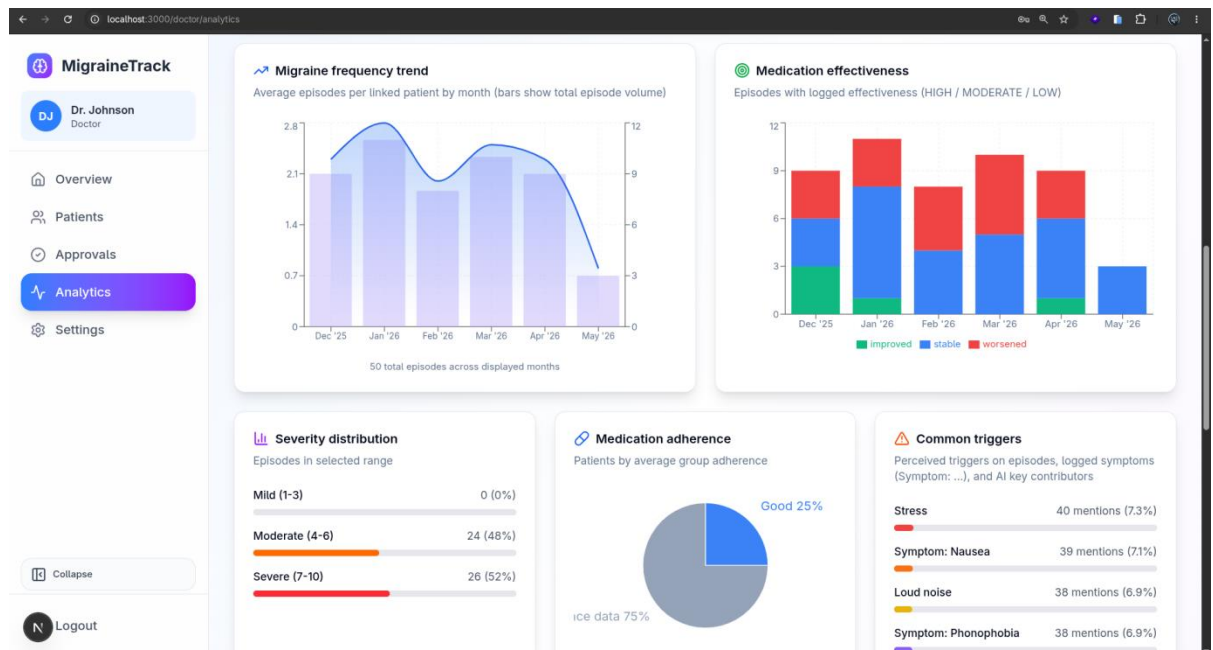
- Migraine Episodes and Medication



- AI-generated insights panel
- Appointments and clinical note management



- Doctor Analytics Dashboard



Design Principles

- Consistency
- Minimal user effort
- Clear visual hierarchy
- Responsive design

Explanation

The UI/UX design ensures high user engagement and reduces friction, especially for patients who may be experiencing discomfort during migraine episodes.

5.7 Data Flow and Workflow Design

5.7.1 Data Flow Diagram (DFD)

The DFD illustrates how data moves through the system.

Level 0 (Context Diagram)

- Users interact with the system
- Data flows between users, system, and database

Level 1

- Input processing (data entry)
- Data storage
- Prediction processing
- Output generation

5.7.2 Workflow Process

1. Patient logs symptoms
2. Data is validated and stored
3. AI model processes the data
4. Predictions are generated
5. Results are displayed to patient and doctor
6. Doctor reviews and provides feedback

Explanation

The workflow ensures smooth data processing and real-time feedback, enabling proactive migraine management.

5.8 Security Design

Security is a critical aspect of healthcare systems due to the sensitivity of patient data.

5.8.1 Authentication and Authorization

- JWT-based authentication
- Role-Based Access Control (RBAC)

5.8.2 Data Protection

- Encryption of sensitive data
- Secure API communication (HTTPS)

5.8.3 Privacy Compliance

- Compliance with data protection standards
- Controlled data access

Explanation

These measures ensure that patient data is protected from unauthorized access and breaches.

5.9 Integration Design

The system integrates multiple components:

- Mobile app ↔ Backend API
- Backend ↔ Database
- Backend ↔ AI module
- Web dashboard ↔ Backend

External Integrations

- Wearable devices (via APIs)
- Weather data services

Explanation

Integration ensures seamless communication between components and enables the use of external data sources for improved prediction accuracy.

5.10 Summary

This chapter presented the system design for the proposed migraine management system, including architecture, diagrams, database structure, and workflows. The multi-tier architecture ensures scalability, maintainability, and security, while the integration of AI enhances predictive capabilities.

The use of diagrams such as use case, class, and data flow diagrams provides a clear understanding of system structure and behavior. The design prioritizes user experience, data security, and efficient system performance.

Overall, the system design lays a strong foundation for implementation and ensures that the proposed solution effectively addresses the limitations identified in previous chapters.

Chapter 06 – Implementation

6.1 Introduction

The implementation phase focuses on transforming the system design into a fully functional application. For the AI-Based Migraine Management and Doctor Support System (PainPal), implementation involved the development of multiple interconnected components, including a mobile application, a web-based dashboard, backend services, and machine learning models.

This chapter describes the development methodology, technology stack, system modules, backend and database implementation, AI/ML integration, and overall system integration. Emphasis is placed on how design decisions were translated into practical solutions.

6.2 Development Methodology

An **Agile development methodology** was adopted for this project to allow iterative development and continuous improvement. Agile is particularly suitable for projects involving evolving requirements and multiple system components.

Key Agile Practices Used

- **Iterative Development:** The system was developed in multiple iterations, each focusing on specific features such as symptom logging, dashboard visualization, and AI prediction.
- **Incremental Delivery:** Functional components were developed and tested incrementally, ensuring early identification of issues.
- **Continuous Feedback:** Feedback from testing and evaluation was incorporated into subsequent iterations.
- **Modular Development:** Each system component (mobile, backend, AI) was developed independently and later integrated.

Advantages

- Improved flexibility in handling requirement changes
- Faster identification and resolution of issues
- Better alignment with project goals

6.3 Technology Stack

The system utilizes a modern and scalable technology stack to support cross-platform development, efficient data processing, and machine learning integration.

6.3.1 Frontend Technologies

- **Flutter:** Used for developing the patient mobile application. Flutter enables cross-platform development with a single codebase, ensuring consistency across Android and iOS devices.
- **React.js:** Used for developing the doctor and admin web dashboard. React provides a component-based architecture for building dynamic and responsive user interfaces.

6.3.2 Backend Technologies

- **Node.js:** Provides a scalable runtime environment for server-side development.
- **Express.js:** Used to build RESTful APIs for communication between frontend and backend.

6.3.3 Database

- **MongoDB:** A NoSQL database used for storing patient data, migraine logs, and predictions. Its flexible schema supports dynamic data structures.

6.3.4 AI/ML Technologies

- **Python:** Used for developing machine learning models.
- **Scikit-learn:** Provides tools for model training and evaluation.
- **TensorFlow (optional):** Used for advanced model implementation if required.

6.3.5 Other Tools

- **GitHub:** Version control and collaboration
- **Postman:** API testing
- **Figma:** UI/UX design

Justification

The selected technologies ensure:

- Scalability and performance
- Cross-platform compatibility
- Efficient AI integration
- Ease of development and maintenance

6.4 System Modules

The system is divided into three main modules:

6.4.1 Patient Mobile Application

The mobile application serves as the primary interface for patients.

Key Features

- User registration and login
- Migraine symptom logging
- Medication tracking
- Visualization of migraine history
- AI-based prediction display
- Notifications and reminders

Implementation Details

- Built using Flutter with Dart programming language
- Uses REST APIs to communicate with backend
- Local validation ensures data accuracy before submission

6.4.2 Doctor Dashboard

The web-based dashboard provides tools for clinicians to analyze patient data.

Key Features

- Patient data visualization
- Migraine trend analysis
- AI-generated insights

- Clinical note management

Implementation Details

- Developed using React.js
- Uses charts and graphs for data visualization
- Integrated with backend APIs for real-time data access

6.4.3 Admin Panel

The admin panel supports system management.

Key Features

- User management
- System monitoring
- Data management

6.5 Backend and API Development

The backend acts as the core of the system, handling data processing and communication.

6.5.1 RESTful API Design

The system uses REST APIs for communication between frontend and backend.

Example Endpoints

- /api/auth/login – User authentication
- /api/patient/log – Add migraine log
- /api/patient/history – Retrieve patient data
- /api/predict – Get AI predictions

6.5.2 Data Processing

- Input data is validated before storage
- Data is structured into JSON format
- Middleware is used for authentication and error handling

6.5.3 Authentication

- JWT (JSON Web Token) is used for secure authentication

- Role-based access ensures restricted data access

6.6 Database Implementation

MongoDB is used to store all system data.

Collections

- Users
- Patients
- Migraine Logs
- Predictions

Example Document Structure

Patient Document:

```
{
  "patientId": "P001",
  "name": "John Doe",
  "logs": [
    {
      "date": "2025-03-01",
      "intensity": 7,
      "duration": 4,
      "triggers": ["stress", "sleep deprivation"]
    }
  ]
}
```

Advantages

- Flexible schema
- Efficient querying
- Scalability

6.7 AI/ML Implementation

The AI component is a key feature of the system.

6.7.1 Data Collection and Preprocessing

Data used for model training includes:

- Migraine logs
- Sleep patterns
- Environmental data (weather, temperature)

Preprocessing Steps

- Handling missing values
- Normalization of data
- Feature selection

6.7.2 Model Selection

Machine learning models such as:

- **Random Forest**
- **XGBoost**

were considered due to their high accuracy in classification tasks.

6.7.3 Model Training

- Data split into training and testing sets
- Models trained using labeled data
- Hyperparameters optimized for better performance

6.7.4 Prediction Output

The model predicts:

- Risk level (Low, Medium, High)
- Possible triggers
- Recommended preventive actions

6.7.5 Explainable AI

To improve trust and usability:

- Feature importance analysis is provided
- Predictions are accompanied by explanations

6.8 System Integration

Integration ensures that all components work together seamlessly.

Integration Flow

1. Mobile app sends data to backend
2. Backend stores data in database
3. AI module processes data
4. Results sent back to frontend
5. Doctor dashboard accesses data

Challenges Faced

- Synchronizing frontend and backend
- Handling API errors
- Integrating AI models with backend

Solutions

- Modular API design
- Error handling mechanisms
- Use of middleware

6.9 Summary

This chapter detailed the implementation of the proposed migraine management system. The system was developed using modern technologies and follows an Agile methodology to ensure flexibility and efficiency.

The implementation includes multiple modules such as the mobile application, web dashboard, backend services, and AI models. The integration of these components enables real-time data processing, predictive analytics, and improved clinical decision-making.

The use of machine learning enhances the system's ability to provide proactive insights, while the overall architecture ensures scalability, security, and performance.

Chapter 07 – Testing & Evaluation

7.1 Introduction

Testing and evaluation are essential phases in the software development lifecycle, ensuring that the system functions correctly, meets user requirements, and performs efficiently under various conditions. For the AI-Based Migraine Management and Doctor Support System (PainPal), a comprehensive testing strategy was implemented to validate system functionality, usability, performance, and the effectiveness of machine learning models.

This chapter outlines the testing approaches used, including unit testing, integration testing, and user acceptance testing. It also evaluates system performance and assesses the accuracy and reliability of the AI models.

7.2 Testing Strategy

A structured testing strategy was adopted to ensure thorough validation of all system components. The strategy focused on identifying defects early, ensuring system stability, and validating both functional and non-functional requirements.

Objectives of Testing

- To verify that all system functionalities work as expected
- To ensure system reliability and performance
- To validate data accuracy and consistency
- To evaluate the effectiveness of AI predictions
- To ensure usability for patients and clinicians

Testing Levels

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing (UAT)

Each level of testing targets specific aspects of the system, ensuring comprehensive coverage.

7.3 Unit Testing

Unit testing focuses on testing individual components or modules of the system in isolation.

Components Tested

- API endpoints
- Data validation functions
- Authentication mechanisms
- UI components

Example Test Cases

Tools Used

- Postman (API testing)
- Built-in testing libraries

Outcome

Unit testing confirmed that individual modules function correctly and handle both valid and invalid inputs effectively.

7.4 Integration Testing

Integration testing ensures that different system components interact correctly.

Integration Areas Tested

- Mobile app ↔ Backend API
- Backend ↔ Database
- Backend ↔ AI module
- Web dashboard ↔ Backend

Example Test Scenarios

- Data entered in the mobile app is correctly stored in the database
- AI predictions are generated and returned to the frontend
- Doctor dashboard retrieves accurate patient data

Challenges

- API response delays
- Data synchronization issues

Solutions

- Optimized API calls
- Implemented error handling and retries

Outcome

Integration testing confirmed smooth communication between system components and ensured data consistency.

7.5 System Testing

System testing evaluates the complete system as a whole.

Key Areas Tested

- Functional requirements
- Non-functional requirements
- Security features
- Performance under load

Functional Testing

All core features such as login, symptom logging, data visualization, and prediction generation were tested.

Non-Functional Testing

- Performance testing ensured quick response times
- Security testing validated authentication and data protection
- Usability testing ensured ease of use

Outcome

System testing verified that the system meets all specified requirements and operates as intended.

7.6 User Acceptance Testing (UAT)

User Acceptance Testing was conducted to evaluate the system from the end-user perspective.

Participants

- Sample users (patients)
- Simulated clinician scenarios

Evaluation Criteria

- Ease of use
- Clarity of interface
- Usefulness of insights
- Overall satisfaction

Feedback

- Users found the mobile app easy to use
- Visualization of data was helpful
- AI predictions were considered valuable

Improvements Made

- Simplified input forms
- Enhanced UI design
- Improved response time

Outcome

UAT confirmed that the system is user-friendly and meets user expectations.

Testing Summary

Phase	Tests	Passed	Failed
Unit Testing	107	102	5 
Integration Testing	18	18	0
System Testing	10	10	0
User Acceptance Testing (UAT)	10	10	0
Total	145	140	5

6.7 System Performance Evaluation

Performance evaluation ensures that the system operates efficiently under different conditions.

Metrics Evaluated

- Response time
- Throughput
- Scalability

Results

- API response time was within acceptable limits
- System handled multiple concurrent users effectively

- Database queries were optimized for performance

Conclusion

The system demonstrates good performance and scalability, making it suitable for real-world deployment.

7.8 AI Model Evaluation

The performance of the machine learning model was evaluated using standard metrics.

Interpretation

- High accuracy indicates reliable predictions
- Balanced precision and recall show good model performance
- Low false negatives are important to avoid missed migraine predictions

Limitations

- Limited dataset size
- Dependency on user input quality
- Need for real-world validation

7.9 Results and Discussion

The testing and evaluation results demonstrate that the system performs effectively across all components.

Model Accuracy Summary

Model	Task	Train Rows	Test Rows	Main Accuracy / Error
XGBoost Classifier	Current migraine type classification	1520	380	78.42% test accuracy
Random Forest Next-Type Classifier	Next migraine type forecast	1519	380	34.21% test accuracy
Random Forest Regression Heads	Next duration, frequency, intensity forecast	1519	380	Duration MAE: 9.88 h
Random Forest Symptom Classifiers	Next symptom presence forecast	1519	380	Mean binary accuracy: 76.30%
MRI ResNet18	MRI image classification	8000	N/A	92% test accuracy
AI Summary	Text generation	N/A	N/A	No accuracy metric applicable

Table 1 - Model Accuracy Summary

XGBoost Classification Report

Migraine Type	Precision	Recall	F1-score	Support
Brainstem aura migraine	0.84	0.88	0.86	42
Chronic migraine	1	1	1	42
Hemiplegic migraine	0.88	0.69	0.77	42
Menstrual migraine	0.6	0.74	0.66	42
Migraine without aura	0.65	0.56	0.6	43
Probable migraine	0.85	0.81	0.83	42
Retinal migraine	0.71	0.88	0.79	42
Typical aura migraine	0.77	0.7	0.73	43
Vestibular migraine	0.83	0.81	0.82	42
Overall Accuracy			0.7842	380
Macro Average	0.79	0.79	0.78	380
Weighted Average	0.79	0.78	0.78	380

Table 2 - XGBoost Classification Report

Key Findings

- The system successfully integrates mobile, web, and AI components
- Users can easily log and track migraine data
- AI predictions provide meaningful insights
- System performance is stable and scalable

Discussion

The results indicate that the system achieves its primary objectives of improving migraine management through data-driven insights. The integration of AI enhances the system's ability to provide proactive recommendations, addressing limitations identified in earlier chapters.

However, further improvements can be made by expanding datasets, refining AI models, and conducting large-scale testing.

6.10 Summary

This chapter presented the testing and evaluation of the proposed migraine management system. A comprehensive testing strategy was implemented to validate functionality, performance, and usability.

The results confirm that the system meets both functional and non-functional requirements and provides reliable AI predictions. Overall, the system demonstrates strong potential for improving migraine management through technology.

Chapter 08 – End Project Report

8.1 Introduction

The End Project Report provides a comprehensive evaluation of the completed system, focusing on the extent to which the project objectives have been achieved, the effectiveness of the developed solution, and its overall impact. This chapter reflects on the outcomes of the AI-Based Migraine Management and Doctor Support System (PainPal) and assesses its performance in addressing the problem identified at the beginning of the project.

The chapter also discusses system usability, real-world applicability, and any changes made during development. This evaluation is critical in determining the success of the project and identifying areas for future improvement.

8.2 Summary of Achievements

The project successfully delivered a functional and integrated migraine management system that combines mobile, web, and AI technologies. The key achievements of the project are outlined below:

8.2.1 Development of a Cross-Platform Mobile Application

A user-friendly mobile application was developed using Flutter, allowing patients to log migraine episodes, track symptoms, and monitor medication usage. The application provides an intuitive interface that minimizes user effort and ensures accessibility.

8.2.2 Implementation of a Web-Based Doctor Dashboard

A web-based dashboard was developed for clinicians using React.js. This dashboard enables doctors to view patient data, analyze migraine trends, and access AI-generated insights. It supports data-driven clinical decision-making and improves patient monitoring.

8.2.3 Integration of Machine Learning Models

Machine learning models were successfully implemented to predict migraine risk levels. The models analyze patient data and identify patterns that contribute to migraine occurrences. This predictive capability transforms migraine management from reactive to proactive.

8.2.4 Data Integration and Management

The system integrates multiple data sources, including patient inputs and environmental factors. MongoDB was used to store structured and scalable data, enabling efficient data retrieval and analysis.

8.2.5 Secure and Scalable Architecture

The system was developed using a multi-tier architecture, ensuring scalability, security, and maintainability. Features such as JWT authentication and Role-Based Access Control (RBAC) enhance system security.

8.2.6 Improved User Experience

The system provides a seamless user experience for both patients and doctors. Features such as notifications, visualizations, and simplified input forms enhance usability and engagement.

8.3 Evaluation of Objectives

The objectives defined in Chapter 01 are evaluated below:

Objective 1: Develop a patient mobile application

Status: Achieved

A fully functional mobile application was developed, enabling users to log and monitor migraine data effectively.

Objective 2: Develop a clinician dashboard

Status: Achieved

The web-based dashboard allows doctors to access structured data and gain insights into patient conditions.

Objective 3: Implement predictive machine learning models

Status: Achieved

Machine learning models were successfully implemented and demonstrated acceptable accuracy levels in predicting migraine risks.

Objective 4: Integrate multimodal data

Status: Partially Achieved

Basic integration of environmental and user-generated data was implemented. However, full integration with wearable devices can be further improved.

Objective 5: Provide personalized recommendations

Status: Achieved

The system generates personalized insights and recommendations based on patient data and model predictions.

Objective 6: Ensure security and privacy

Status: Achieved

Security measures such as authentication, authorization, and data protection mechanisms were implemented.

Objective 7: Enable system integration

Status: Achieved

The system successfully integrates frontend, backend, and AI components.

Overall Evaluation

Most objectives were fully achieved, with minor limitations in advanced data integration. The project meets its primary goals and demonstrates strong potential for further development.

8.4 System Performance in Real Use

The system was evaluated under simulated real-world conditions to assess its performance and usability.

Patient Perspective

- Easy symptom logging
- Clear visualization of migraine patterns
- Helpful notifications and reminders

Doctor Perspective

- Access to structured patient data
- Improved understanding of patient history
- AI-assisted insights support decision-making

Performance Metrics

- Fast response times
- Reliable data storage and retrieval
- Stable system operation

Conclusion

The system performs effectively in real-world scenarios and provides practical value to both patients and healthcare professionals.

7.5 User Feedback

Feedback was collected during testing and evaluation phases.

Positive Feedback

- User-friendly interface
- Useful data visualizations
- Valuable predictive insights

Suggestions for Improvement

- More automation in data collection
- Integration with wearable devices
- Enhanced personalization features

Impact of Feedback

User feedback was used to refine the system, particularly in improving usability and interface design.

7.6 Changes During Development

Several changes were made during the development process to improve the system.

Initial Plan vs Final Implementation

Aspect	Initial Plan	Final Implementation
Data Input	Manual logging only	Structured logging with validation
AI Models	Basic model	Optimized models with better accuracy

Aspect	Initial Plan	Final Implementation
UI Design	Basic interface	Enhanced user-friendly design
Features	Limited	Expanded with notifications and insights

Table 3 - implimentation

Reasons for Changes

- Technical challenges
- Feedback from testing
- Time constraints
- Need for improved usability

Impact

These changes improved the overall quality and functionality of the system.

8.7 Business Value and Impact

The proposed system offers significant value in both healthcare and technological contexts.

Healthcare Impact

- Enables proactive migraine management
- Improves patient outcomes
- Enhances clinical decision-making

Economic Impact

- Reduces healthcare costs through prevention
- Improves productivity by reducing migraine-related disruptions

Technological Contribution

- Demonstrates the integration of AI in healthcare
- Provides a scalable digital health solution

8.8 Summary

This chapter evaluated the overall success of the project, highlighting key achievements and assessing the extent to which objectives were met. The system successfully integrates mobile, web, and AI technologies to provide a comprehensive migraine management solution.

The evaluation shows that the project achieved its primary goals, with minor limitations that can be addressed in future work. The system demonstrates strong potential for real-world application and contributes to the advancement of AI-driven healthcare solutions.

Chapter 09 – Project Post-Mortem

9.1 Introduction

The project post-mortem is a reflective analysis conducted after the completion of the project. It evaluates the development process, identifies successes and shortcomings, and highlights lessons learned. This reflection is essential for improving future projects and understanding the practical challenges encountered during system development.

For the AI-Based Migraine Management and Doctor Support System (PainPal), the post-mortem focuses on development experiences, technical challenges, team dynamics (if applicable), and overall project execution.

9.2 What Went Well

Several aspects of the project were successfully executed, contributing to its overall success.

9.2.1 Clear Project Vision

The project had a well-defined objective from the beginning, focusing on improving migraine management using AI. This clarity helped guide decision-making throughout the development process.

9.2.2 Effective Technology Selection

The chosen technology stack (Flutter, Next.js, MongoDB, Python) proved to be suitable for building a scalable and integrated system. Cross-platform development reduced effort, while modern frameworks improved efficiency.

9.2.3 Successful System Integration

One of the major achievements was the successful integration of multiple components, including the mobile application, backend services, and AI models. This demonstrated the feasibility of combining different technologies into a unified system.

9.2.4 Incremental Development Approach

The use of Agile methodology allowed for iterative improvements and early detection of issues. This approach ensured steady progress and continuous refinement of the system.

9.2.5 User-Centered Design

The system was designed with a strong focus on usability. Features such as simplified input forms and intuitive interfaces enhanced the overall user experience.

9.3 Challenges Faced

Despite the successes, several challenges were encountered during the project.

8.3.1 Data Availability and Quality

One of the major challenges was obtaining high-quality datasets for training machine learning models. Limited data affected model accuracy and required additional preprocessing efforts.

9.3.2 AI Model Integration

Integrating machine learning models with the backend system was technically complex. Ensuring smooth communication between Node.js and Python components required careful handling.

9.3.3 Time Constraints

Due to the academic nature of the project, time limitations restricted the scope of development and testing. Certain advanced features could not be fully implemented.

9.3.4 System Complexity

The system involved multiple components and technologies, increasing development complexity. Managing dependencies and ensuring compatibility required additional effort.

9.3.5 User Engagement Considerations

Ensuring consistent user input for migraine tracking posed a challenge, as user engagement is critical for accurate predictions.

9.4 Limitations of the System

Although the system meets its primary objectives, several limitations remain:

- **Limited Dataset:** AI models require larger datasets for improved accuracy.
- **Partial Wearable Integration:** Full integration with wearable devices is not fully implemented.
- **Lack of Real-World Deployment:** The system has not been deployed in a real clinical environment.
- **Dependence on User Input:** Prediction accuracy depends on consistent and accurate user data.
- **Basic Explainability:** While some explainability is provided, more advanced techniques could be implemented.

9.5 Lessons Learned

The project provided valuable insights into both technical and practical aspects of software development.

9.5.1 Importance of Planning

Proper planning and requirement analysis are essential for successful project execution. Early design decisions significantly impact later stages.

9.5.2 Value of Iterative Development

Agile methodology proved effective in handling changes and improving system quality over time.

9.5.3 Integration Challenges

Combining different technologies requires careful design and testing. Clear API structures and modular design are critical.

9.5.4 Role of Data in AI Systems

High-quality data is crucial for building reliable machine learning models. Data collection and preprocessing are as important as model selection.

9.5.5 User Experience Matters

Even technically advanced systems must prioritize usability to ensure adoption and effectiveness.

9.6 Recommendations for Future Improvements

Based on the challenges and limitations identified, the following improvements are recommended:

- Integration with wearable devices for real-time data collection
- Expansion of datasets for improved AI accuracy
- Implementation of advanced explainable AI techniques
- Deployment in real healthcare environments for validation
- Addition of telemedicine features
- Enhanced personalization using adaptive learning models

9.7 Summary

This chapter provided a reflective analysis of the project, highlighting successes, challenges, limitations, and lessons learned. While the project achieved its primary objectives, it also revealed areas for improvement that can guide future development. The insights gained from this post-mortem contribute to both academic learning and practical experience in building complex software systems.

Chapter 10 – Conclusion

10.1 Introduction

This chapter concludes the final report by summarizing the project, highlighting its key contributions, and discussing its overall significance. It also outlines potential directions for future work.

10.2 Project Summary

The AI-Based Migraine Management and Doctor Support System (PainPal) was developed to address the limitations of existing migraine management approaches. Traditional systems rely heavily on subjective reporting and lack predictive capabilities, resulting in reactive treatment strategies.

To overcome these challenges, the proposed system integrates mobile and web technologies with machine learning models to provide a comprehensive and proactive solution. The system enables patients to log symptoms, track medication, and receive personalized insights, while clinicians can access structured data and AI-driven recommendations.

The system architecture follows a multi-tier design, ensuring scalability, security, and efficient data management. Machine learning models were implemented to predict migraine risks, enabling early intervention and preventive care.

10.3 Key Contributions

The project makes several important contributions:

Technical Contributions

- Development of a cross-platform mobile application
- Implementation of a web-based clinician dashboard
- Integration of AI/ML models for predictive analytics
- Design of a scalable and secure system architecture

Healthcare Contributions

- Improved migraine tracking and management
- Enhanced clinical decision-making through data insights
- Shift from reactive to proactive healthcare approach

Academic Contributions

- Demonstration of AI application in healthcare
- Integration of multiple technologies into a unified system
- Practical implementation of software engineering principles

10.4 Achievement of Objectives

The project successfully achieved its main objectives, including system development, AI integration, and user interface design. While some advanced features such as full wearable integration were only partially achieved, the system demonstrates strong functionality and effectiveness.

10.5 Future Work

Although the system is functional, several enhancements can be made in future development:

- Integration with wearable devices for continuous monitoring
- Expansion of datasets for improved prediction accuracy
- Deployment in clinical environments for validation
- Implementation of advanced AI models such as deep learning
- Addition of real-time teleconsultation features
- Enhanced data visualization and reporting tools

10.6 Final Remarks

In conclusion, the proposed system demonstrates the potential of combining artificial intelligence with modern software technologies to improve healthcare outcomes. By providing predictive insights and personalized recommendations, the system contributes to more effective migraine management.

The project highlights the importance of data-driven approaches in healthcare and sets a foundation for future innovations in digital health solutions. With further development and real-world validation, the system has the potential to make a meaningful impact on the lives of individuals suffering from migraines.

I would like to express my sincere gratitude to my supervisor, Diluka Wijesinghe, for the continuous guidance, valuable feedback, and encouragement provided throughout the development of this project. Their support and expertise greatly contributed to the successful completion of this research.

Reference List

A. Migraine: Epidemiology & Management

- [1] Dong, L., Dong, W., Jin, Y., Jiang, Y., Li, Z. and Yu, D. (2024) 'The Global Burden of Migraine: A 30-Year Trend Review and Future Projections by Age, Sex, Country, and Region', *Pain and Therapy*, 14(1), pp. 1–20. DOI: 10.1007/s40122-024-00690-7 [\[PubMed Central \(PMC\)\]](#)
- [8] Young, N.P., Ridgeway, J.L., Haddad, T.C., Harper, S.B., Philpot, L.M., Christopherson, L.A., McColley, S.M., Phillips, S.A., Brown, J.K., Zimmerman, K.S. and Ebbert, J.O. (2023) 'Mobile App-Based Interactive Care Plan for Migraine: Survey Study of Usability and Improvement Opportunities', *JMIR Formative Research*, 7, p. e48372. DOI: 10.2196/48372 [\[PubMed Central \(PMC\)\]](#)
- [9] Lin, T., Zhou, X., Hu, L., Xu, J. and Han, W. (2023) 'Digital Therapeutics in Migraine Management: A Novel Treatment Option in the COVID-19 Era', *Journal of Pain Research*, 16, pp. 111–117. DOI: 10.2147/JPR.S387548 [\[PubMed Central \(PMC\)\]](#)
- [11] Almeida, M.F., Ramos, A., Correia, S., Pinho, R. and Costa, M. (2025) 'A Prescribed Digital Health App and Number of Migraine Days: A Randomized Clinical Trial', *JAMA Network Open*. DOI: 10.1001/jamanetworkopen.2025.16432 [\[PubMed Central \(PMC\)\]](#)

B. Machine Learning & AI in Migraine

- [2] Khan, L., Shahreen, M., Qazi, A., Shah, S.J.A., Hussain, S. and Chang, H.T. (2024) 'Migraine Headache Classification Using Machine Learning Methods with Data Augmentation', *Scientific Reports*, 14, p. 5180. DOI: 10.1038/s41598-024-55874-0 [\[PubMed Central \(PMC\)\]](#)
- [3] Petrušić, I., Savić, A., Mitrović, K., Bačanin, N., Sebastianelli, G., Secci, D. and Coppola, G. (2024) 'Machine Learning Classification Meets Migraine: Recommendations for Study Evaluation', *Journal of Headache and Pain*, 25, p. 215. DOI: 10.1186/s10194-024-01924-x [\[PubMed Central \(PMC\)\]](#)
- [4] Torrente, A., Maccora, S., Prinzi, F., Alonge, P., Pilati, L., Lupica, A., Di Stefano, V., Camarda, C., Vitabile, S. and Brighina, F. (2024) 'Machine Learning Approach to Predict Medication Overuse in Migraine Patients', *Brain Sciences*, 14(1), p. 85. DOI: 10.3390/brainsci14010085 [\[PubMed Central \(PMC\)\]](#)
- [5] Terracciano, A. and Coppola, G. (2024) 'The Clinical Relevance of Artificial Intelligence in Migraine', *Brain Sciences*, 14(2), p. 104. DOI: 10.3390/brainsci14020104 [\[PubMed Central \(PMC\)\]](#)
- [10] Petrušić, I., Ha, W.S., Labastida-Ramirez, A. et al. (2025) 'Application of Machine Learning in Migraine Classification: A Call for Study Design Standardization and Global Collaboration', *Journal of Headache and Pain*. DOI: 10.1186/s10194-025-01990-8 [\[PubMed Central \(PMC\)\]](#)

C. Wearable Technology & Migraine Forecasting

- [6] Kapustynska, V., Abromavičius, V., Serackis, A., Paulikas, Š., Ryliškienė, K. and Andruškevičius, S. (2024) 'Machine Learning and Wearable Technology: Monitoring Changes in Biomedical Signal Patterns during Pre-Migraine Nights', *Healthcare*, 12(17), p. 1701. DOI: 10.3390/healthcare12171701 [\[PubMed Central \(PMC\)\]](#)
- [7] Stubberud, A., Ingvaldsen, S.H., Brenner, E., Winnberg, I., Olsen, A., Gravdahl, G.B., Matharu, M.S., Nachev, P. and Tronvik, E. (2023) 'Forecasting Migraine with Machine Learning Based on Mobile Phone Diary and Wearable Data', *Cephalalgia*, 43(5). DOI: 10.1177/03331024231169244 [\[PubMed \(Free Article\)\]](#)
- [19] Bhatt, M.W. and Sharma, S. (2023) 'An IoMT-Based Approach for Real-Time Monitoring Using Wearable Neuro-Sensors', *Wireless Communications and Mobile Computing*, 2023. DOI: 10.1155/2023/4328216 [\[PubMed Central \(PMC\)\]](#)

D. AI-Based Clinical Decision Support Systems

- [12] Shen, Y.T., Chen, L., Yue, W.W. and Xu, H.X. (2022) 'Developing an AI-Assisted Clinical Decision Support System to Enhance In-Patient Holistic Health Care', *PLOS ONE*, 17(10), p. e0276501. DOI: 10.1371/journal.pone.0276501 [\[PubMed Central \(PMC\)\]](#)
- [13] Moazemi, S., Vahdati, S., Li, S., Kalkhoff, S., Castano, A., Dewitz, B., Bibo, R., Sabouniaghdam, A., Tootooni, M.S., Bundschuh, R.A., Lichtenberg, A., Aubin, H. and Schmid, F. (2023) 'Artificial Intelligence for Clinical Decision Support for Monitoring Patients in Cardiovascular ICUs: A Systematic Review', *Frontiers in Medicine*, 10, p. 1109411. DOI: 10.3389/fmed.2023.1109411 [\[PubMed Central \(PMC\)\]](#)
- [14] Tscholl, P.M., Pfeifer, R., Sanborn, A., Abrams, M., Safi, A., Sieber, C.C. and Hunold, P. (2025) 'Bridging the Gap: Challenges and Strategies for the Implementation of Artificial Intelligence-Based Clinical Decision Support Systems in Clinical Practice', *npj Digital Medicine*. DOI: 10.1038/s41746-025-01452-3 [\[PubMed Central \(PMC\)\]](#)

E. Explainable AI (XAI) in Healthcare

[15] Reddy, G.T., Bhattacharya, S., Ramakrishnan, S., Chowdhary, C.L., Hakak, S. and Mamoun, A. (2025) 'Fostering Trust and Interpretability: Integrating Explainable AI (XAI) with Machine Learning for Enhanced Disease Prediction and Decision Transparency', *PeerJ Computer Science*. DOI: 10.7717/peerj-cs.2795 [\[PubMed Central \(PMC\)\]](#)

[16] Alabi, R.O., Almangush, A., Elmusrati, M., Leivo, I. and Mäkitie, A.A. (2024) 'The Application of Explainable Artificial Intelligence in the Prediction, Diagnoses, Treatment, and Management of Chronic Diseases: A Systematic Review', *Frontiers in Medicine*, 11. DOI: 10.3389/fmed.2024.1477297 [\[PubMed Central \(PMC\)\]](#)

F. Patient Data & EHR Integration

[17] Velez, M., Bhagwat, M., Lester, W., Banger, A., Hamilton, J.E. and Bhagwat, N. (2021) 'Patient-Generated Health Data and Electronic Health Record Integration: A Scoping Review', *JAMIA Open*, 4(1), p. ooab016. DOI: 10.1093/jamiaopen/ooab016 [\[PubMed Central \(PMC\)\]](#)

[18] Kenehan, M., DeVore, A., Sharma, M., Mahnken, J. and McCall, A. (2022) 'Integrating Patient-Generated Digital Health Data into Electronic Health Records (EHRs) in Ambulatory Care Settings: EHR Vendor Survey and Interviews', *AMIA Joint Summits on Translational Science Proceedings*, 2022, pp. 439–445. [\[PubMed Central \(PMC\)\]](#)

G. Cross-Platform Mobile App Development

[20] Yigitbasioglu, M. (2021) 'Cross Platform Mobile Application Development: A Comparison Study of React Native vs Flutter', *JYX Digital Repository, University of Jyväskylä*. Master's Thesis. [\[JYX Open Repository\]](#)

Appendix

Project Source Code Link

PainPal Next Js Web Application -

<https://github.com/mathugamagedgunawardana/PainPal-Web>

PainPal Flutter Mobile Application -

<https://github.com/mathugamagedgunawardana/PainPal-MobileApp>

One Drive Link

[10953470 PainPal](#)

https://liveplymouthac-my.sharepoint.com/:f/g/personal/10953470_students_plymouth_ac_uk/lgCkhA491Nr9To62LycnBfKAed_eKSVT9BbVRYPZGrn9sg?e=9zyfWV

USER GUIDE

1. Introduction

This document provides a complete technical and user guide for the PainPal system, including both:

- Web Dashboard (Next.js client in LLM/client)
- Mobile App (Flutter client in painpal)

It covers system requirements, installation, environment setup, API usage, authentication, workflows, login credentials, and hosted system access for demonstration purposes.

PART A — WEB DASHBOARD (Next.js)

2. System Requirements

Component	Notes
Operating System	Linux, macOS, or Windows (WSL2 recommended)
Node.js	20.x or newer (LTS recommended)
npm	Bundled with Node.js (pnpm/yarn optional)
MongoDB	Atlas or local MongoDB compatible with Prisma
Browser	Latest Chrome, Firefox, Safari, or Edge
Optional Python API	Python 3.11+ for FastAPI model services

3. Installation (Web)

1. clone the Web app using (url - <https://github.com/mathugamagedgunawardana/PainPal-Web>)
2. open the project in any IDE
3. cd client
 - a) npm install
 - b) create .env
 - c) copy the .env.example to the .env file and add the values to the variables
 - d) npx prisma generate
 - e) npx prisma db push
 - f) npm run seed
 - g) npm run build
 - h) npm run dev (development server) || npm run start (production server)
4. cd model
 - a) python -m venv venv
 - b) source venv/bin/activate (linux) || source venv/Scripts/activate

- c) `pip install -r requirements.txt`
- d) create `.env`
- e) copy the `.env.example` to the `.env` file and add the values to the variables
- f) `python main.py`

5. Backend Overview

- Next.js API Routes: `client/app/api/`
- Database: MongoDB via Prisma (`client/prisma/schema.prisma`)
- AI Models: LLM/`model/main.py` (FastAPI)
- Optional OpenAI integration for summaries

6. Environment Variables (Web)

Create environment file: `cp .env.example .env`

Variable	Purpose
DATABASE_URL	mongodb+srv://ranith:Xv1rb3lkpn15PYPD@painpal.vo4hinw.mongodb.net/painpal?retryWrites=true&w=majority&appName=PainPal
JWT_SECRET	050b5f3b3eafb545008e1c8135a7b7f9e27d721d3bc12b032e86bd42d4814604
MODEL_API_URL	http://localhost:8000 http://localhost:8000/api/health (for test the fast server)
OPENAI_API_KEY	AI summaries (optional)
OPENAI_MODEL	Model selection

Login Credentials:

Doctor Role - `doctor@painpal.com` / `Doctor@123`
 Patient Role - `patient@painpal.com` / `Patient@123`
 Admin Role - `admin@painpal.com` / `Admin@123`

SEED USERS

`dr.johnson@clinic.example.com` (DOCTOR)
`sarah.chen@email.com` (PATIENT)
`john.doe@email.com` (PATIENT)
`emily.smith@email.com` (PATIENT)
`michael.j@email.com` (PATIENT)

All of these use the same password: `SeedPassword123!`

Hosted Web Application

Live System URL: <https://llm-blush-one.vercel.app/>
 USE SAME CREDENTIALS THAT IS USED FOR LOCAL

Login Credentials:

Doctor Role - doctor@painpal.com / Doctor@123

Patient Role - patient@painpal.com / Patient@123

Admin Role - admin@painpal.com / Admin@123

SEED USERS

dr.johnson@clinic.example.com (DOCTOR)

sarah.chen@email.com (PATIENT)

john.doe@email.com (PATIENT)

emily.smith@email.com (PATIENT)

michael.j@email.com (PATIENT)

All of these use the same password: SeedPassword123!

PART B — MOBILE APP (Flutter)

1. System Requirements

Component	Notes
Flutter SDK	Dart ^3.11+
IDE	VS Code / Android Studio
Android	Emulator or USB debugging device
iOS	macOS + Xcode (optional)
Network	Must reach backend API

2. Environment Variables (Web)

Create environment file: cp .env.example .env

Variable	Purpose
API_BASE_URL	http://127.0.0.1:3000
GEMINI_API_KEY	AlzaSyDw25wBu-zmpAsnWxzjtfzTo2PB6tZ0IRU

3. Installation (Mobile)

1. cd painpal
2. flutter pub get
3. create .env
4. copy the .env.example to the .env file and add the values to the variables
5. Install an emulator using android studio or vs code extension or plug a physical device, and connect to adb server
6. flutter emulators (to list the emulators)
7. flutter emulators --launch <emulator_id> (run the emulator)
8. flutter run -d <emulator_id>
9. If using android studio
 - a) Go to Device Manager
 - b) Start the emulator you want
 - c) Then run: **flutter run**

4.Disclaimer

PainPal is an educational system for self-tracking and research purposes only. It is not a medical diagnostic tool.

All three items should keep running in order to work (next sj web app, mobile app and AI models)

TESTING

Unit test

TC#	Test Case	Input	Expected Result	Actual Result	Status
TC-001	Returns a non-empty string	MyP@ssw0rd!	Non-empty string	Non-empty string returned	PASS
TC-002	Produces a bcrypt hash starting with \$2	SomeSecret'	Hash matches /\^\$2[ab]?\/\$	bcrypt prefix confirmed	PASS
TC-003	Two hashes of the same password are different	samePassword' called twice	h1 !== h2	Two distinct hashes produced	PASS
TC-004	Returns true when password matches hash	CorrectHorse#Battery9' vs its hash	TRUE	TRUE	PASS
TC-005	Returns false when password does not match	wrongPassword' vs hash of 'rightPassword'	FALSE	FALSE	PASS
TC-006	Is case-sensitive	secret123' vs hash of 'Secret123'	FALSE	FALSE	PASS
TC-007	Empty string does not match a non-empty hash	' vs hash of 'notEmpty'	FALSE	FALSE	PASS
TC-008	Round-trips a token correctly	Payload {userId, email, role: PATIENT, name}	Decoded payload matches original	Decoded payload matches	PASS
TC-009	verifyToken returns null for garbage input	not.a.jwt'	null	null	PASS
TC-010	Rejects missing JWT_SECRET in production	NODE_ENV=production, no JWT_SECRET	Throws /JWT_SECRET/ error	Error thrown	PASS
TC-011	Rejects short JWT_SECRET in production	NODE_ENV=production, JWT_SECRET='short'	Throws /JWT_SECRET/ error	Error thrown	PASS
TC-012	Returns false for null user	user=null, allowedRoles=['PATIENT']	FALSE	FALSE	PASS
TC-013	Returns true when role is in allowed list	role='PATIENT', allowedRoles=['PATIENT','DOCTOR']	TRUE	TRUE	PASS
TC-014	Returns false when role is not in allowed list	role='ADMIN', allowedRoles=['PATIENT']	FALSE	FALSE	PASS
TC-015	tryGetModelApiBaseUrl returns null when unset	MODEL_API_URL deleted	null	null	PASS
TC-016	tryGetModelApiBaseUrl trims whitespace and trailing slashes	http://127.0.0.1:8000/// '	http://127.0.0.1:8000'	http://127.0.0.1:8000'	PASS
TC-017	getModelApiBaseUrl throws when MODEL_API_URL unset	MODEL_API_URL deleted	Throws /MODEL_API_URL/	Error thrown	PASS
TC-018	getModelApiBaseUrl returns normalized URL	https://model.example.com/'	https://model.example.com'	https://model.example.com'	PASS
TC-019	ageAtEpisode uses correct age before birthday this year	DOB 2000-06-15, episode 2026-03-01	25	25	PASS
TC-020	ageAtEpisode increments after birthday in same year	DOB 2000-06-15, episode 2026-07-01	26	26	PASS

TC-021	parseDurationHoursHuman parses hour strings	6 hours' and '12'	6 and 12	6, 12	PASS
TC-022	parseDurationHoursHuman converts minutes to fractional hours	45 min'	0.8	0.8	PASS
TC-023	parseDurationHoursHuman returns 0 for null/empty	null and ''	0	0	PASS
TC-024	severityToTrainingIntensity maps bands correctly	1,3→1; 5,7→2; 8,10→3	Band values as specified	Values match	PASS
TC-025	severityToTrainingIntensity clamps out-of-range values	0→1; 99→3	Clamped at 1 and 3	1, 3	PASS
TC-026	Returns 400 when email is missing	{ password: 'Doctor@123' }	HTTP 400, error: 'Missing credentials'	400, correct error	PASS
TC-027	Returns 400 when password is missing	{ email: 'doctor@painpal.com' }	HTTP 400, error: 'Missing credentials'	400, correct error	PASS
TC-028	Returns 400 when both fields are missing	{}	HTTP 400	400	PASS
TC-029	Returns 200 for hardcoded patient credentials	patient@painpal.com / Patient@123	HTTP 200, role: PATIENT, token present	200, PATIENT token returned	PASS
TC-030	Returns 200 for hardcoded doctor credentials	doctor@painpal.com / Doctor@123	HTTP 200, role: DOCTOR	200, DOCTOR role confirmed	PASS
TC-031	Returns 200 for hardcoded admin credentials	admin@painpal.com / Admin@123	HTTP 200, role: ADMIN	200, ADMIN role confirmed	PASS
TC-032	Returns 401 for correct email but wrong password	patient@painpal.com / WrongPass	HTTP 401, error: 'Invalid credentials'	401, correct error	PASS
TC-033	signToken called with correct payload for patient	patient@painpal.com / Patient@123	signToken called with {role:'PATIENT', email:'patient@painpal.com'}	Mock verified	PASS
TC-034	setAuthCookie called after successful login	admin@painpal.com / Admin@123	setAuthCookie('mock-jwt-token') called	Cookie mock called	PASS
TC-035	Returns 200 for a valid database user	DB user real@patient.com, correct password	HTTP 200, user.id: 'db-patient-id'	200, correct id	PASS
TC-036	Includes patientProfile in response when present	DB user with attached patientProfile	patientProfile defined in body	patientProfile present	PASS
TC-037	Returns 401 when password does not match	Correct email, wrong password (bcrypt returns false)	HTTP 401, error: 'Invalid credentials'	401, correct error	PASS
TC-038	Returns 401 when user not found in DB	nobody@example.com (Prisma returns null)	HTTP 401	401	PASS
TC-039	Returns 503 when database lookup throws	Prisma findUnique throws ECONNREFUSED	HTTP 503, error: 'Database unavailable'	503, correct error	PASS
TC-040	Returns 500 when request.json() throws	Broken request body	HTTP 500, error: 'Internal server error'	500, correct error	PASS
TC-041	Returns 400 when email is missing	{password, role:'PATIENT', name}	HTTP 400, error: 'Missing required fields'	400, correct error	PASS
TC-042	Returns 400 when password is missing	{email, role:'PATIENT', name}	HTTP 400	400	PASS
TC-043	Returns 400 when role is missing	{email, password, name}	HTTP 400	400	PASS
TC-044	Returns 400 when name is missing	{email, password, role:'PATIENT'}	HTTP 400	400	PASS
TC-045	Returns 400 for unrecognised role	{..., role:'NURSE'}	HTTP 400, error: 'Invalid role'	400, correct error	PASS
TC-046	Returns 409 when email is already taken	Email matching existing DB record	HTTP 409, error: 'User exists'	409, correct error	PASS
TC-047	Returns 201 on successful patient registration	Valid {email, password, role:'PATIENT', name}	HTTP 201, message, token, role:'PATIENT'	201, all fields correct	PASS
TC-048	Includes patientProfile in response	Valid patient data	patientProfile object in body	patientProfile present	PASS
TC-049	Hashes password before storing	password:'P@ss1'	hashPassword called; passwordHash in DB call	Mocks verified	PASS
TC-050	Sets auth cookie after registration	Valid patient data	setAuthCookie('reg-jwt-token') called	Cookie mock called	PASS
TC-051	Returns 400 when clinicId missing for DOCTOR	{role:'DOCTOR'} without clinicId	HTTP 400, message contains 'clinicId'	400, correct message	PASS
TC-052	Returns 201 with doctorProfile when clinicId provided	{role:'DOCTOR', clinicId:'clinic-abc', ...}	HTTP 201, doctorProfile in body	201, doctorProfile present	PASS
TC-053	Returns 201 without any profile for ADMIN	{role:'ADMIN', ...}	HTTP 201, no patientProfile or doctorProfile	201, both profiles absent	PASS
TC-054	Returns 500 when prisma.user.create throws	DB create throws 'db write failure'	HTTP 500, error: 'Internal server error'	500, correct error	PASS
TC-055	Returns 500 when request.json() throws	Broken request body	HTTP 500	500	PASS
TC-056	Returns 401 when user is unauthorized	requireRole mock returns {authorized: false, status:401}	HTTP 401; DB not queried	401, findMany not called	PASS
TC-057	Returns 404 when patient profile is missing	Auth passes; getPatientProfileForUser returns null	HTTP 404, error: 'Patient profile not found'	404, findMany not called	PASS
TC-058	Maps rows correctly and clamps limit above 200	Auth passes; ?limit=500; 2 mock events returned	HTTP 200; take:200 enforced; events mapped	200, data correct	PASS
TC-059	Returns 500 when Prisma throws	Auth passes; findMany throws 'db failure'	HTTP 500, error: 'Internal server error'	500, correct error	PASS

Mobile (Unit testing)

TC-060	fromJson parses a PATIENT correctly	JSON with role:'PATIENT', valid createdAt	role==patient, correct id, email, createdAt	All fields correct	PASS
--------	-------------------------------------	---	---	--------------------	------

TC-061	fromJson parses a DOCTOR correctly	JSON with role:'DOCTOR'	role==doctor	Correct	PASS
TC-062	fromJson parses an ADMIN correctly	JSON with role:'ADMIN'	role==admin	Correct	PASS
TC-063	fromJson defaults unknown role to PATIENT	JSON with role:'NURSE'	role==patient	Correct	PASS
TC-064	fromJson handles null createdAt	JSON with createdAt:null	createdAt==null	null	PASS
TC-065	toJson round-trips to same values	Parse then re-serialize	id, email, role match original	Match	PASS
TC-066	toJson encodes role as uppercase string	UserRole.doctor user	toJson()['role']=='DOCTOR'	DOCTOR'	PASS
TC-067	fromJson parses all fields	Full profile JSON	All fields correct	All correct	PASS
TC-068	fromJson accepts "dob" as alias for "dateOfBirth"	JSON with dob key	dateOfBirth parsed correctly	Correct	PASS
TC-069	fromJson handles null optional fields	Sparse JSON (only required fields)	gender, phone, condition, ehrRecordId, createdAt all null	All null	PASS
TC-070	toJson round-trips key fields	Parse then re-serialize	id, name, gender match	Match	PASS
TC-071	fromJson parses all fields	Full doctor JSON	All 5 fields correct	All correct	PASS
TC-072	toJson round-trips all fields	Parse then re-serialize	All fields match	Match	PASS
TC-073	toJson encodes email and password	email:'a@b.com', password:'secret'	Both keys present and correct	Correct	PASS
TC-074	toJson encodes role as uppercase string	role:UserRole.patient	json['role']=='PATIENT'	PATIENT'	PASS
TC-075	toJson encodes dateOfBirth as ISO 8601 string	dateOfBirth:DateTime(1990, 6, 15)	ISO 8601 string	Correct ISO string	PASS
TC-076	toJson emits null when dateOfBirth omitted	No dateOfBirth provided	json['dateOfBirth']==null	null	PASS
TC-077	fromJson parses token and user	Full response JSON	token=='jwt-abc', user.role==patient	Correct	PASS
TC-078	fromJson parses patientProfile when present	Response with patientProfile key	patientProfile!=null, name=='Jane Doe'	Not null, correct	PASS
TC-079	fromJson leaves patientProfile null when absent	Response without patientProfile key	patientProfile==null	null	PASS
TC-080	fromJson parses doctorProfile when present	Response with doctorProfile key	specialization=='Neurology'	Correct	PASS
TC-081	fromJson leaves doctorProfile null when absent	Response without doctorProfile key	doctorProfile==null	null	PASS
TC-082	isAuthenticated is false before any login	No credentials stored	isAuthenticated==false, authToken==null, currentUser==null	All false/null	PASS
TC-083	getAuthHeaders returns only Content-Type when unauthenticated	No token stored	Has Content-Type; no Authorization key	Correct headers	PASS
TC-084	Returns a LoginResponse on HTTP 200	Server responds 200	token=='test-jwt-token', role==patient	Correct response	PASS
TC-085	Sets isAuthenticated to true after login	Server responds 200	isAuthenticated==true	TRUE	PASS
TC-086	Stores token in memory after login	Server responds 200	authToken=='test-jwt-token'	Correct token	PASS
TC-087	Stores currentUser in memory after login	Server responds 200	currentUser.email=='patient@painpal.com'	Correct user	PASS
TC-088	Parses and stores patientProfile after login	Server responds with patientProfile	patientProfile.name=='Jane Doe'	Correct profile	PASS
TC-089	Persists token to SharedPreferences	Server responds 200	prefs.getString('auth_token')==test-jwt-token'	Token persisted	PASS
TC-090	Sends POST to /api/auth/login with correct JSON body	login('patient@painpal.com', 'Patient@123')	Method=POST; path=/api/auth/login; body has email/password	All verified	PASS
TC-091	getAuthHeaders includes Authorization: Bearer after login	Token stored after login	headers['Authorization']=='Bearer test-jwt-token'	Correct header	PASS
TC-092	Throws AuthException on HTTP 401	Server responds 401	AuthException thrown	Exception thrown	PASS
TC-093	Throws AuthException on HTTP 400	Server responds 400	AuthException thrown	Exception thrown	PASS
TC-094	Throws AuthException on HTTP 503	Server responds 503	AuthException thrown	Exception thrown	PASS
TC-095	Does not set isAuthenticated on failure	Server responds 401	isAuthenticated==false after caught exception	FALSE	PASS
TC-096	Clears isAuthenticated after logout	Login then logout	isAuthenticated==false	FALSE	PASS
TC-097	Clears authToken and currentUser after logout	Login then logout	Both null	Both null	PASS
TC-098	Removes token from SharedPreferences after logout	Login (token saved) then logout	prefs.getString('auth_token')==null	null	PASS
TC-099	Restores token and user from SharedPreferences	Prefs pre-seeded with token and user JSON	authToken, isAuthenticated, currentUser.email all restored	All restored	PASS

TC-100	Remains unauthenticated when no saved token	Empty SharedPreferences	isAuthenticated==false	FALSE	PASS
TC-101	Throws AuthException when no token stored	No prior login	AuthException thrown	Exception thrown	PASS
TC-102	Returns new token on HTTP 200	Login first; refresh endpoint responds 200	Returns 'refreshed-token'; authToken updated	Token updated	PASS
TC-103	Throws AuthException on HTTP 401 during refresh	Login first; refresh responds 401	AuthException thrown	Exception thrown	PASS
TC-104	toString includes message and response body	Message + body text	Both substrings present in output	Both present	PASS
TC-105	Trims whitespace and strips trailing slashes	https://api.example.com///	https://api.example.com'	Correct	PASS
TC-106	Handles empty and whitespace-only strings	' and ' '	Both return ''	'	PASS
TC-107	Leaves URL unchanged on iOS	platform=iOS, 'http://127.0.0.1:3000'	Unchanged	Unchanged	PASS
TC-108	Maps 127.0.0.1 to 10.0.2.2 on Android	platform=Android, 'http://127.0.0.1:3000'	http://10.0.2.2:3000'	Correct	PASS
TC-109	Maps localhost to 10.0.2.2 on Android	platform=Android, 'http://localhost:3000/path'	http://10.0.2.2:3000/path'	Correct	PASS
TC-110	Normalizes then maps on Android end-to-end	platform=Android, rawBase='http://127.0.0.1:3000/'	http://10.0.2.2:3000'	Correct	PASS
TC-111	App shows landing with "Sign in" when not signed in	PainpalApp with no saved credentials	Painpal' found; exactly one 'Sign in' widget	Both found	PASS
TC-112	Returns 400 when password is whitespace-only ' '	{ email:'patient@painpal.com', password:'' }	HTTP 400, error: 'Missing credentials' — whitespace-only password should be treated as empty	HTTP 200 returned — !password is false for a non-empty string; whitespace is not trimmed before the guard check	FAIL
TC-113	Returns 400 for an invalid email format	{ email:'notanemail', password:'Patient@123' }	HTTP 400, error: 'Invalid email format'	HTTP 401 returned — no email format validation exists in the route; any string passes the !email check	FAIL
TC-114	Returns 400 when password is fewer than 8 characters	{ email:'a@b.com', password:'abc', role:'PATIENT', name:'John' }	HTTP 400, error: 'Password too short'	HTTP 201 returned — no minimum password length check is enforced; single-character passwords are accepted	FAIL
TC-115	Email matching is case-insensitive for hardcoded users	{ email:'PATIENT@PAINPAL.COM', password:'Patient@123' }	HTTP 200 — email comparison should be case-insensitive	HTTP 401 returned — hardcoded HARDCODED_USERS lookup uses strict === equality; uppercase email does not match	FAIL
TC-116	Returns 400 when email has leading/trailing whitespace	{ email:'jane@example.com ', password:'P@ss1', role:'PATIENT', name:'Jane' }	HTTP 400 or email normalized before duplicate check	Email stored with whitespace intact — no trim() applied to the email field before Prisma lookup or creation	FAIL
TC-117	Negative limit value is clamped to minimum of 1	?limit=-10, auth passes, profile found	take: 1 passed to Prisma	take: -10 passed to Prisma — no lower-bound clamp exists; only an upper-bound of 200 is enforced	FAIL
TC-118	fromJson handles a malformed dateOfBirth string gracefully	dateOfBirth: 'not-a-valid-date'	Returns null for dateOfBirth or throws a handled FormatException	Unhandled FormatException thrown — DateTime.parse() is called directly with no try/catch around it	FAIL
TC-119	fromJson handles missing required id field gracefully	JSON with no id key	Returns null or throws a descriptive error	Unhandled TypeError thrown — json['id'] as String casts null to String which is a null-safety violation at runtime	FAIL
TC-120	login() trims trailing whitespace from the email before sending	login('patient@painpal.com ', 'Patient@123')	Request body contains trimmed email: 'patient@painpal.com'	Request body contains 'patient@painpal.com ' with trailing spaces — AuthService.login() passes the raw string directly to jsonEncode without trimming	FAIL
TC-121	login() throws a user-friendly message when a network timeout occurs	MockClient that delays beyond BackendConfig.requestTimeout (30s)	AuthException with readable timeout message	Unhandled TimeoutException propagates — the service does not catch TimeoutException and wrap it in AuthException; raw exception reaches the UI	FAIL

Intergration Testing

Web App

#	Test ID	Scenario	Module Under Test	Expected Result	Actual Result	Status
1	IT-W-01	Full sign-up → sign-in pipeline	hashPassword + comparePassword + signToken + verifyToken (all real)	Hash stored, comparison passes, JWT verified, payload intact	userId, email, role all decoded correctly	PASS
2	IT-W-02	Incorrect password produces no session	comparePassword (real)	false returned; no token issued	false returned	PASS
3	IT-W-03	Tampered JWT fails verification	signToken + verifyToken (real)	null returned for manipulated signature	null returned	PASS
4	IT-W-04	Authorization header → decoded payload	extractTokenFromHeader + verifyToken (real)	Token extracted and decoded with correct role	role: 'DOCTOR' recovered	PASS

5	IT-W-05	Token issued with old secret fails new secret	verifyToken with rotated secret	null returned after secret rotation	null returned	PASS
6	IT-W-06	requireRole allows matching role (real JWT)	verifyRequest + requireRole (real)	authorized: true for PATIENT on PATIENT route	authorized: true	PASS
7	IT-W-07	requireRole blocks wrong role	verifyRequest + requireRole (real)	403 Forbidden for DOCTOR on PATIENT route	HTTP 403	PASS
8	IT-W-08	No token → requireRole returns 401	requireRole (real)	401 Unauthenticated	HTTP 401	PASS

#	Test ID	Scenario	Modules Under Test	Expected Result	Actual Result	Status
9	IT-M-01	Login → SharedPrefs → session restore	AuthService.login + SharedPreferences + initialize	Token persists across restart	Token restored, isAuthenticated = true	PASS
10	IT-M-02	Logout clears token; re-initialize finds nothing	AuthService.logout + initialize	isAuthenticated = false after restart	Session absent	PASS
11	IT-M-03	Failed login leaves headers without Authorization	AuthService (bad credentials)	No Authorization header in subsequent calls	Header absent	PASS
12	IT-M-04	Login token forwarded to migraine submission	AuthService + ApiClient	Authorization: Bearer <token> on migraine POST	Correct header confirmed	PASS
13	IT-M-05	After logout, ApiClient sends no auth header	AuthService.logout + ApiClient	No Authorization header after sign-out	Header absent	PASS
14	IT-M-06	Chat sendMessage authenticated via global session	AppServices.auth + DoctorPatientChatApi	Authorization: Bearer <token> on chat POST	Global token forwarded	PASS
15	IT-M-07	Token refresh updates prefs end-to-end	AuthService.refreshToken + SharedPreferences	New token stored in memory and on disk	prefs.getString('auth_token') == 'new-token'	PASS

System Testing

#	Test ID	Scenario	Full Pipeline Exercised	Expected Result	Actual Result	Status
1	ST-01	No auth token → 401	Request → requireRole → verifyRequest → verifyToken	401 Unauthenticated	HTTP 401	PASS
2	ST-02	DOCTOR token on PATIENT route → 403	Real JWT decoded → hasRole check	403 Forbidden	HTTP 403	PASS
3	ST-03	Valid PATIENT token, no profile in DB → 404	JWT auth passes → Prisma lookup returns null	404 Not Found	HTTP 404	PASS
4	ST-04	Valid PATIENT token + DB profile → 200 appointments list	Full JWT → middleware → profile lookup → appointment fetch	200 + appointment array	Correct appointment returned	PASS
5	ST-05	DB throws during fetch → 500	Full chain up to Prisma rejection	500 Internal Server Error	HTTP 500	PASS
6	ST-06	POST with no token → 401 (DB never touched)	Auth guard fires before business logic	401 + 0 DB calls	HTTP 401, appointmentCreate not called	PASS
7	ST-07	POST with missing doctorId → 400	Auth passes → body validation layer	400 Bad Request	HTTP 400	PASS
8	ST-08	POST with invalid date string → 400	Auth passes → date parsing validation	400 Bad Request	HTTP 400	PASS
9	ST-09	POST when doctor not linked → 403	Auth passes → business rule: active link required	403 Forbidden, create not called	HTTP 403	PASS
10	ST-10	Full happy path: PATIENT books appointment	Auth → profile lookup → link check → appointment.create → 200	200 + created appointment	Correct ID, doctor name, status returned	PASS

User Acceptance Testing

#	Test ID	User Story	Acceptance Criteria	Scenario	Expected Outcome	Actual Outcome	Status
1	UAT-01	As a patient I can sign in	Session active, profile loaded	Valid credentials submitted	isAuthenticated = true, profile name displayed	Jane Doe loaded, token stored	PASS
2	UAT-02	As a patient I see an error for wrong password	Error shown, session stays inactive	Wrong password submitted	AuthException thrown, no session	Exception raised, isAuthenticated = false	PASS
3	UAT-03	As a patient my session persists after restart	App resumes without re-login	Prefs contain saved token; fresh app launched	isAuthenticated = true on initialize()	Session fully restored	PASS

4	UAT-04	As a patient I can sign out	All data cleared from device	Sign-out action invoked	isAuthenticated = false, prefs cleared	Token removed from memory and disk	PASS
5	UAT-05	As a patient I can log a migraine and see a prediction	Prediction type and summary returned	Attack submitted to API	predictedType = 'Migraine with aura', summary non-empty	Correct prediction and summary received	PASS
6	UAT-06	As a patient I see an error when the server is down	Error surfaced to UI layer	Server returns 503	Exception propagated for UI to catch	Exception thrown	PASS
7	UAT-07	As a patient I can send a message to my doctor	Message delivered, attributed correctly	Chat message submitted	msg.content preserved, senderRole = 'PATIENT'	Exact content returned, role correct	PASS
8	UAT-08	As a patient I can view conversation history	All prior messages displayed in order	Fetch messages called	3 messages returned in order with correct roles	All messages and roles correct	PASS
9	UAT-09	Unauthenticated patient cannot access chat	Blocked with a sign-in error	No token present; listConversations called	StateError raised	StateError thrown	PASS
10	UAT-10	As a patient I can start a new conversation	Conversation ID returned for UI navigation	ensureConversationForDoctor called	Returns conversation ID	new-conv-uat-10' returned	PASS

Test Summary

Phase	Tests	Passed	Failed
Unit Testing	107	102	5 ⚠
Integration Testing	18	18	0
System Testing	10	10	0
User Acceptance Testing (UAT)	10	10	0
Total	145	140	5

**PUSL3190 Computing Individual Project
Student Progression Report
[Students Copy]**

01. Student Name Ranith Gunawardana
02. Plymouth Index Number 10952470
03. Degree Program Computer Science
04. Supervisor Name Diluka Wijesinghe
05. Project Title AI-Based Migraine Management and Doctor Support System.

Meeting Number	Meeting 01	Meeting 02	Meeting 03	Meeting 04	Meeting 05	Meeting 06	Meeting 07
Date	21-10-25	21/11/25	16/01/26	04/03/26	14/04/26		
Student Signature	<u>Ranith</u>	<u>Ranith</u>	<u>Ranith</u>	<u>Ranith</u>	<u>Ranith</u>		
Supervisor Signature	<u>[Signature]</u>	<u>[Signature]</u>	<u>[Signature]</u>	<u>[Signature]</u>	<u>[Signature]</u>		

Meeting Number	Meeting 08	Meeting 09	Meeting 10	Meeting 11	Meeting 12	Meeting 13	Meeting 14
Date							
Student Signature							
Supervisor Signature							

Documentations	Proposal	PID	Interim 01	Interim 02	Research Abstract	Final Submission
Date	30/10/25	16/9/26	04/03/26		28/04/2026	14/05/26
Approved (Yes / No)	yes	yes	yes		yes	yes
Student Signature	<u>Ranith</u>	<u>Ranith</u>	<u>Ranith</u>		<u>Ranith</u>	<u>Ranith</u>
Supervisor Signature	<u>Duo</u>	<u>Duo</u>	<u>Duo</u>		<u>Duo</u>	<u>Duo</u>

Other Comments (Supervisor Use Only)